Hellenic Air Force Academy

Department of Aeronautical Sciences

Division: Electronics, Electrical Power,

Telecommunications

Diploma Thesis

"Recognition and automatic calculation of the number

of people entering a building"

IK IV (ME) Rizos Konstantinos

Supervisor: Assistant Professor

Pavlatos Christos

Evaluated by the three-member examination committee

On…….../……../…….

Committee Chair Supervising Professor

Member

Dekelia 2022

……

Rizos Konstantinos

Ikaros IV Mechanical Electronic Engineer

With reservation of all rights. All rights reserved.

Copying, storing and distributing this work, in whole or in part, for commercial purposes is prohibited. Reprinting, storing and distributing for non-commercial, educational or research purposes is allowed, provided the source is cited and this notice is preserved. Inquiries regarding the use of the work for commercial purposes should be directed to the author.

The views and conclusions contained in this document express the author and should not be interpreted as representing the official positions of the

Calvary Wings School

Dedication

I would like to dedicate this current diploma thesis to my family which supported me not only during the conduct of the work through advice and suggestions for its improvement, but also for the support I received during the duration of my studies at the Icarian School.

TABLE OF CONTENTS

SUMMARY

ABSTRACT

ABSTRACT

SUMMARY

CHAPTER CONTENTS

ACKNOWLEDGEMENTS

LIST OF PHOTOGRAPHS

LIST OF TABLES

LIST OF FIGURES

PREFACE

PART I

THEORY

CHAPTER 1 INTRODUCTION

CHAPTER 2 THEORETICAL BACKGROUND

PART II

SYSTEM AND APPLICATION

CHAPTER 3 ARCHITECTURE OF THE PROPOSED SYSTEM

CHAPTER 4 SYSTEM HARDWARE

CHAPTER 5 SYSTEM SOFTWARE

CHAPTER 6 SYSTEM PRESENTATION

CHAPTER 7 CONCLUSIONS - FUTURE EXTENSIONS

BIBLIOGRAPHY

APPENDIX I SYSTEM CODE

Summary

One of the most significant modern problems plaguing society on a global scale, is the coronavirus, a deadly virus that appeared on December 8, 2019. The coronavirus has pushed society to its limits and has caused enormous economic destruction to our planet.

Delving deeper, we observe the major problem that has been caused to the Armed Forces, with an active force of 110,000 1, consisting of combat and non-combat personnel. Military operations have been significantly affected, while the problem of overcrowding is one of the biggest issues the Armed Forces have faced in the last 2 years.

To combat the virus, it is deemed necessary to take appropriate measures to prevent its spread. The most modern monitoring and recognition systems can play a decisive role in addressing the problem of personnel overcrowding. The development of a modern detection and monitoring system requires technological knowledge from different research fields such as machine learning, communications, and computer science, including hardware, software, and algorithms.

The science of machine learning, Deep Learning, can be used to monitor an area of interest and automatically check at any time the number of personnel present in the area. With the development of this system, counting is facilitated, its accuracy is increased, and the problem of overcrowding and supervision is addressed.

1https://el.wikipedia.org/wiki/%CE%95%CE%BB%CE%BB%CE%B7%CE%BD%CE%B9%CE%BA%CE%AD%CF%82_%CE%88%CE%BD%CE%BF%CF%80%CE%BB%CE%B5%CF%82_%CE%94%CF%85%CE%BD%CE%AC%CE%BC%CE%B5%CE%B9%CF%82
(Accessed May 18, 22)

Abstract

This thesis deals with modern and alternative ways of addressing, counteracting and preventing the spread of the coronavirus. With the help of modern electronic and technological means, a program was developed that implemented the solution for the above needs.

The main objective of the work is the construction of a program which The main objective of the work is the construction of a program that is efficient and easily implementable. The ease of implementation entails the minimum possible economic and technological background. The program was implemented and tested on two different systems. The initial system we worked with was the Raspberry Pi 3 system. As will be proven below, however, this system could not support our demanding program and in addition, purchasing such a system is rather expensive. The next application was done with the help of a simple computer, where the results were quite satisfactory. Regarding the success of the system, its construction cost is particularly low, while at the same time the program operates effectively with a far greater performance than the Raspberry Pi system.

As for the required technological knowledge, all the actions that will be needed to create the prerequisite virtual support environment are analyzed below. Next, the code of the program will be presented and there will be parallel analysis and explanation of its commands.

Abstract

This dissertation deals with modern and alternative ways of dealing with, combating and preventing the spread of coronavirus. With the help of modern electronic and technological means, a program was constructed which implemented the solution for the above needs.

The main goal of the work is to build a program, which is effective and easily implemented. Ease of implementation includes the least possible financial and technological background. The program was implemented and tested on two different systems. The original system we worked on was the system with Raspberry Pi 3. But as it will be shown below, this system could not support our demanding program and in addition the purchase of such a system is quite expensive. The next application was made with the help of a simple computer, where the results were quite satisfactory. As for the success of the system, its construction cost is particularly low, while at the same time the program works efficiently with much higher performance than the Raspberry Pi system.

In terms of the necessary technological knowledge, all the actions that will be needed in order to create the prerequisite virtual support environment are analyzed below. Then the program code will be presented and there will be a parallel analysis and explanation of its commands.

Summary

One of the most important modern problems plaguing society worldwide is the corona virus, a deadly virus that appeared on December 8, 2019. The corona virus has tested society to its extremes, while it was a huge economic disaster for our planet. .

Going deeper, we observe the big problem it has caused in the Armed Forces, with an active force of 110,0002, consisting of combat and non-personnel. The military operations have been significantly affected, while the problem of overcrowding is one of the biggest problems faced by the Armed Forces in the last 2 years.

In order to fight the virus, it is necessary to take appropriate measures, which will prevent its spread. State-of-the-art tracking and reconnaissance systems can make a significant contribution to the problem of overcrowding. The development of a modern tracking and tracing system requires technological knowledge from different research areas such as the science of machine learning, communications and computers including hardware, software and algorithms.

The science of machine learning, deep learning, can be used to monitor an area of interest and automatically control at any time the number of staff located within the area. The development of this system facilitates counting, increases its accuracy and combats the problem of overcrowding and surveillance.

2 Source : Hellenic Armed Forces - Wikipedia (Πρόσβαση 18 Μαϊ 22)

TABLE OF CONTENTS

PAGE

ABSTRACTS / ACKNOWLEDGEMENTS 19

LIST OF FIGURES 21

LIST OF TABLES 22

LIST OF SHAPES 22

PROLOGUE 23

FIRST CHAPTER LIST OF FIGURES

INTRODUCTION

1 INTRODUCTION 27

SECOND CHAPTER 31

THEORETICAL BACKGROUND

2.1 INTRODUCTION TO OBJECT DETECTION 31

2.1.1 What is Object Detection 31

2.1.2 How Object Detection Works 33

2.1.3 Algorithms for Attainment 35

2.1.4 Comparison of Algorithms 45

2.2 INTRODUCTION TO OBJECT TRACKING 46

2.2.1 What is Object Tracking 46

2.2.2 How Object Tracking Works 47

2.2.3 Algorithms of Attainment 48

CHAPTER THREE 53

ARCHITECTURE OF THE PROPOSED SYSTEM

3 PRESENTATION OF THE PROPOSED ARCHITECTURE 53

CHAPTER FOUR 55

SYSTEM HARDWARE

4.1 EXISTING HARDWARE 55

4.1.1 IP Cameras 55

4.1.2 Server 57

4.2 PROPOSED EXTENSION 60

4.2.1 Raspberry Pi 60

4.2.2 Extension proposal using Raspberry 63

CHAPTER FIVE 67

SYSTEM SOFTWARE

5.1 INTRODUCTION TO DEEP LEARNING 67

5.2 REQUIRED SOFTWARE 69

5.3 REQUIRED WORK ENVIRONMENT 71

CHAPTER SIX 73

SYSTEM PRESENTATION

6.1 USER GUIDE OF THE PROGRAM 73

6.2 SYSTEM CODE ANALYSIS 85

6.3 SYSTEM RESULT DEMONSTRATION 89

SEVENTH CHAPTER 97

CONCLUSIONS – FUTURE EXTENSIONS

7.1 CONCLUSIONS 97

7.2 FUTURE APPLICATIONS 98

BIBLIOGRAPHY 99

APPENDIX I SYSTEM CODE 101

ACKNOWLEDGEMENTS

A decisive factor in investigating the subject of the present
The completion of this Military Diploma or from a sufficient time contact with assistant professor Mr. Pavlatos Christos. I extend my warmest thanks to my family for their support and interest. Similarly, I extend my warmest thanks especially to my father, who at the very first moment stood by me, devoted personal time and effort to offer any help possible. Without the help and advice of all the above, this diploma thesis would have had a completely different form and structure. In concluding my thanks I want to stress that the credits for the good points of the diploma belong to everyone who helped me, and at this point I feel obliged to thank all who contributed in any way

in completing the present work.

LIST OF PHOTOS

Figure 1.1 Covid-19 27
Figure 1.2 Overcrowding Problem in the Army 28

Figure 2.1 Representation of Object Detection Result 31
Figure 2.2 Design of Object of Interest Frame 33
Figure 2.3 Virtual Representation of Object Detection Operation 35
Figure 2.4 Image Filtering by Neural Networks..... 36
Figure 2.5 Operation of Convolutional Neural Networks 37
Figure 2.6 Example Image 37
Figure 2.7 Image Segmentation 38
Figure 2.8 Region Categorization 38
Figure 2.9 Prediction of Most Likely Anchor Points with non-max suppression SSD 39
Figure 2.10 Example Image..... 40
Figure 2.11 Image Subsegmentation..... 40
Figure 2.12 Segmentation of Key Regions 41
Figure 2.13 Input Image 41
Figure 2.14 Region Processing 42
Figure 2.15 Contribution of CNN to Regions..... 42
Figure 2.16 Prediction of Bounded Boxes 43
Figure 2.17 Operation of Faster R-CNN 44
Figure 2.18 Box Delimitation 44
Figure 2.19 Example of Object Tracking 47
Figure 2.20 Illustration of Object Tracking Operation 48

Figure 4.1 IP Camera 55
Figure 4.2 Illustration of Wired IP Camera Connection 56
Figure 4.3 Illustration of Wireless IP Camera Connection 57
Figure 4.4 Illustration of Client-Server Architecture 58
Figure 4.5 Final Operating System 60
Figure 4.6 Raspberry Pi Logo 61
Figure 4.7 Illustration of Raspberry Pi 3 62
Figure 4.8 Raspberry Pi 3 64
Figure 4.9 Integrated System Using Raspberry..... 65

Figure 5.1 Comparison of Traditional Algorithms with Deep Learning 68

Figure 6.1 Program Code Snippet 73
Figure 6.2 Program Code Snippet 74
Figure 6.3 Program Code Snippet 75
Figure 6.4 Program Code Snippet 75
Figure 6.5 Program Code Snippet 76
Figure 6.6 Program Code Snippet 77
Figure 6.7 Program Code Snippet 78
Figure 6.8 Program Code Snippet 78
Figure 6.9 Program Code Snippet 79

Figure 6.10 Program Code Snippet .....80 Figure 6.11 Program Code Snippet .....80 Figure 6.12 Program Code Snippet .....82 Figure 6.13 Program Code Snippet .....82 Figure 6.14 Program Code Snippet .....83 Figure 6.15 Program Code Snippet .....83 Figure 6.16 Program Code Snippet .....84 Figure 6.17 Program Code Snippet .....85 Figure 6.18 Program User Guide Illustration .....86 Figure 6.19 Program User Guide Illustration .....86 Figure 6.20 Program User Guide Illustration .....87 Figure 6.21 Program User Guide Illustration .....87 Figure 6.22 Program User Guide Illustration .....88 Figure 6.23 Program User Guide Illustration .....88 Figure 6.24 Program User Guide Illustration .....89 Figure 6.25 Program Operation Presentation .....90 Figure 6.26 Program Operation Presentation .....90 Figure 6.27 Program Operation Presentation .....91 Figure 6.28 Program Operation Presentation .....92 Figure 6.29 Program Operation Presentation .....92 Figure 6.30 Program Operation Presentation .....93 Figure 6.31 Program Operation Presentation .....93 Figure 6.32 Program Operation Presentation .....94 Figure 6.33 Program Operation Presentation .....94 Figure 6.34 Program Operation Presentation 95

Figure 6.35 Program Operation Presentation 95

LIST OF TABLES

Table 2.1 Comparison of Object Detection Algorithms 46

Table 4.1 Summary Table of Hardware-Cost with the Computer Method.....60 Table 4.2 Summary Table of Hardware-Cost with the Raspberry Pi Method 65

LIST OF FIGURES

Figure 3.1 Proposed Architecture 53

PROLOGUE

The era we are experiencing represents a difficult period in the passage of the for mankind. Nowadays, daily life has changed, while the way of living and coexisting does not resemble at all what we had become used to. In humanity and in our society, a very demanding and complex task has been assigned, which is none other than our protection and the fight against the coronavirus. This virus, due to its rapid transmission, affected in a very short time the Armed Forces, which operate within the framework of daily contact and interaction of the personnel.

In the Air Force, the element of communication is a priority of us all, as in order to achieve this difficult task that we are daily called to bring to completion, there must be the proper and immediate interaction among the personnel. Some of the activities that constitute the work in the Air Force include proper aircraft maintenance, ground-based flight control, communication between formation aircraft, conducting flights ensured by their excellent operation, etc. All of the above, in order to be completed safely and successfully, require a main tool that is none other than communication.

A modern enemy of communication is the virus, which did not delay to strike

The modern enemy of communication is the virus, which was not slow to strike our country as well. Thus, alternative ways of conducting communication had to be found, so that the work of the Air Force and in general the Armed Forces would remain steadfast. Through the progress of technology in recent years, in combination with the familiarization of personnel with it, technology proved to be a strong ally in the fight and blocking of the virus transmission. Only by adopting personal protection measures can the virus be fought, as collective measures are also needed, which aim to protect the whole. One of these preventive measures is limiting crowding, especially in closed spaces. For crowd control to prevent exceeding a certain number of people to this day, were responsible crews designated and involved in the process. Therefore, the choice of the topic of this diploma thesis “Recognition and automatic counting of the number of entrants to a building” was carried out according to modern protection requirements and safeguarding from the transmission of the coronavirus, which is an obstacle to the execution of the work of the Armed Forces, whether it concerns a camp or the productive schools which are characterized by the daily living of the cadets and NCOs. Upon completion of this work, an automatic system will be constructed, which as an aid to all our efforts, will contribute to our safeguarding from transmission. This, of course, will be achieved without the involvement of personnel, which until now counted the people entering a space for a lecture, presentation, reception etc., since our primary goal is to approach day by day a daily life that we had become accustomed to before the appearance of the disease. »

PART 1

THEORY

FIRST CHAPTER

INTRODUCTION

1 INTRODUCTION

The virus that plagues the entire planet as well as our country in recent years, in recent years, it brought changes to our daily life, created new realities for humans and the environment. The new everyday life was unknown to humanity and adapting to it proved difficult. The adaptation of people to the new data, to the new way of life, in turn created a multitude of problems that had to be solved.

The World Health Organization carries out daily superhuman efforts to reduce and limit the spread of the virus. The mandatory protective measures imposed by all countries of the world have troubled public opinion; nevertheless, no one can say that protective measures do not work. On the other hand, it is observed that prevention and our personal protection significantly reduce the transmission of the virus. One of the protective measures that we will deal with in this thesis is the control of crowding in closed or open spaces.

Figure 1.1 Covid-19

The virus did not delay long to enter the spaces of the armed forces, as from the very first moment the first cases appeared in Greece, virus cases were announced also in military camps. The space of a camp requires great attention, because it is characterized by crowding and daily contact of the officers and staff. Especially in productive schools, where the confined trainees live most of the week inside the camp, together with hundreds of other people, it is the best hotspot

transmission of the virus, after they sleep, eat and live an intense daily life increasing the risk of transmission to the others, in a short period of time.

Figure 1.2 Congestion Problem in the Army

Thus, the need arises to control the staff and the educates, regarding crowding and adherence to the measures. Since the trainees attend daily academic lessons and lectures in lecture halls, it is important to take protective measures and to designate a specific number of people who can enter high-risk spaces, such as enclosed areas. As for the classes where the lessons take place, it is easy to determine the number of people attending at the same time, as people are divided into sections and each section occupies a separate classroom. When sizes expand and it becomes difficult to record the number of people inside a lecture hall or inside a building, a different solution must be found, which will naturally be characterized by its reliability as well as its simplicity, without involving staff in order to perform the counting.

In this era of modern technology, there is a resource that we have in abundance: a large number of structured and unstructured data. In the second half of the 20th century, machine learning evolved as a subfield of artificial intelligence (AI) that included self-learning algorithms that extract knowledge from data to make predictions. By giving computers the ability to learn from data instead of requiring humans to manually derive rules and build models from analyzing large volumes of data, machine learning offers a more effective alternative for the

capturing knowledge in the data for the gradual improvement of performance of prediction models and data-driven decision making[3]. .

The aim of this thesis, therefore, is the creation of such an electronic system, which will serve the new needs. The choice of a technological means to solve such a problem implies the non-involvement of personnel in the process, but at the same time ensures accurate and reliable measurement, without the result being affected by the human factor of error.

[3] Sebastian Raschka & Vahif Mirjalili, 2019, Python Machine Learning, Published by Packt Publishing Ltd.

SECOND CHAPTER

THEORETICAL BACKGROUND

2.1 INTRODUCTION TO OBJECT DETECTION

2.1.1 What is Object Detection

Object detection, or Object Detection, is a computer vision technique that operates to locate objects within an image or a video4. Through this technology, the computer image is processed appropriately with the aim of recognizing semantic objects of a specific category (such as people, buildings or cars)5. The evolution of Object Detection focused mainly on the areas of face recognition, recognition of people and tracking of video of interest regions.

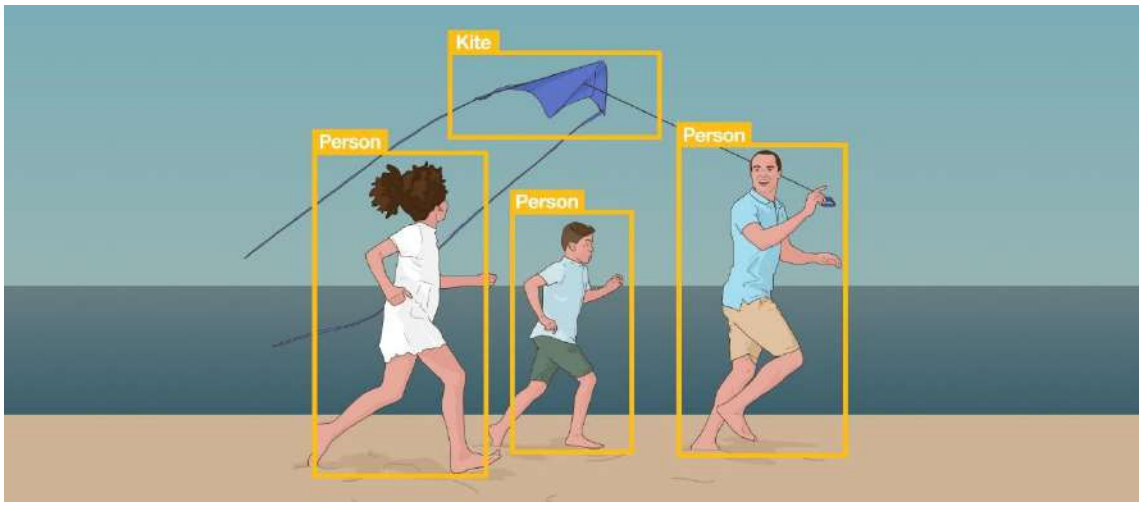

Figure 2.1 Representation of Object Detection result

Object/target tracking refers to the problem of using sensor measurements to determine the position, trajectory and features of objects of interest. A sensor can be be any measuring device, such as radar, sonar, camera, sensor perimeters, microphone, ultrasound or any other sensor that can be used to collect information about objects in the environment. Typical objectives of object tracking are the determination of the number of objects, their identities and states, such as positions,

4 https://www.mathworks.com/discovery/object-detection.html (Accessed May 18, 22)
5 https://en.wikipedia.org/wiki/Object_detection (Accessed May 18, 22)

the speeds and in some cases their features. A characteristic an example of object / target tracking is tracking aircraft with radar. The object tracking problem in this frame attempts to determine the number of aircraft in an area under surveillance, their types, such as military, commercial or transportation, their identities, velocities, and positions. All are based on measurements taken from radar[6].

There is frequent confusion between object detection and image recognition. Therefore, before proceeding, it is important to understand their concepts separately. .

Image recognition assigns a label-title to an image. For example, a photo of a dog receives the label "dog", but a photo of two dogs still receives the label "dog".

On the other hand, object detection draws a bounding box around each deteected object and labels it accordingly. For the above example, it will draw a box around each dog, which box will label as "dog". The model predicts where each object is located and which label should be applied.

It is therefore evident that object detection provides more information about an image compared to recognition, while it can detect multiple objects. .

6 Subhash Challa, Mark R. Morelande, Darko Musicki, Robin J. Evans, 2011, Fundamentals of Object Tracking, Published in the United States of America by Cambridge Press

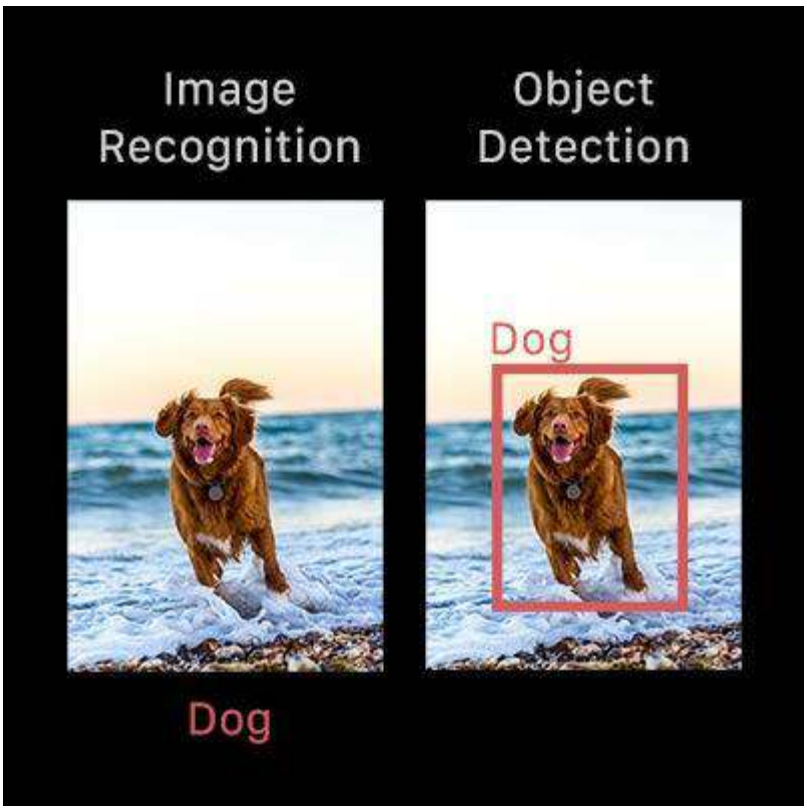

Figure 2.2 Design of object of interest frame

2.1.2 How Object Detection Works

Today there are various models for achieving Object Detection, with the most popular being Haar Cascades and algorithms based on Deep Learning such as the Single Shot Detector and the R-CNN family (R-CNN, Fast R-CNN, Faster R-CNN), which we will analyze below. Machine learning (Deep Learning) has become one of the most exciting technologies of our time. Large companies, such as Google, Facebook, Apple, Amazon, and IBM, are heavily investing in machine learning research and applications. This exciting field is opening the way to new possibilities and has become essential in our daily lives. These include voice assistants on smartphones, credit card fraud prevention, incoming email filtering, medical disease detection and diagnosis, and the list goes on[7].

Object detection models based on Deep Learning usually have two parts. Initially, an encoder takes an image as input and runs it through a series of blocks and layers that learn to extract statistical features used for the detection and

7 Sebastian Raschka & Vahif Mirjalili, 2019, Python Machine Learning, Published by Packt Publishing Ltd

object localization. Then, the outputs of the encoder pass to a decoder, which predicts the bounds and labels for each object. The simplest decoder is a simple recurrent network. The recurrent network is connected to the encoder's output and directly predicts the position and the size of each bounded box. The output of the detection model is the pair of coordinates of the object and its extent in the image. Although simple, this type of model is limited. The number of boxes must be predefined. If the image has two dogs, but the model is designed to detect only one object, one dog will remain without detection, label, and box. However, if the number of objects to be detected in each image is known beforehand, the models based on simple regression can be a good choice.

An extension of the regression approach is a region proposal network. This time, the decoder proposes regions in the image where it believes an object may be located. The pixels that belong to these regions are then fed into a subnetwork for classification to determine a label, if the proposal is not rejected. Then, it passes through a classification network the pixels of the proposed areas that have been approved. The advantage of this method is a more accurate, flexible model that can propose arbitrary numbers of regions that may contain a bounding box. The added accuracy, however, has the cost in computational efficiency, which implies a more demanding and time-consuming system.[8]

8 https://www.fritz.ai/object-detection/ (Access May 18, 22)

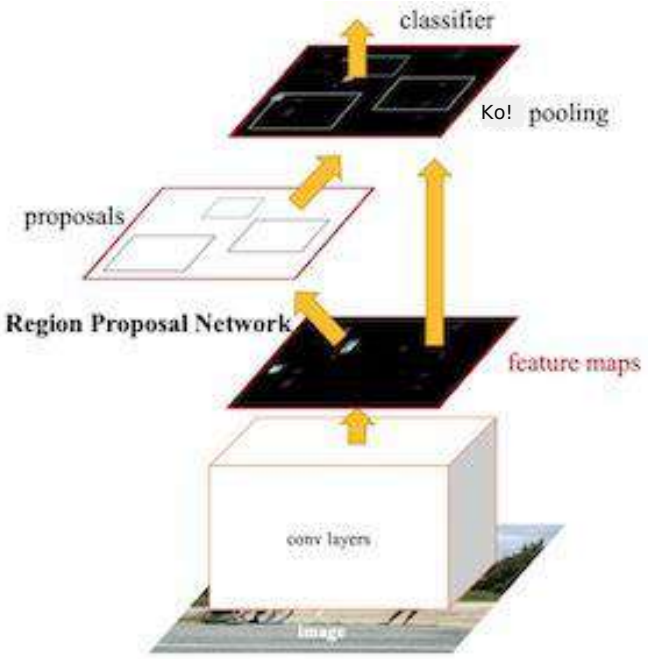

Figure 2.3 Virtual representation of the functioning of Object Detection

2.1.3 Achievement Algorithms

Over time, various Object Detection algorithms. Each algorithm has its advantages, but also has its difficulties depending on how slow and demanding it is. A brief analysis of the main algorithms.

To understand the algorithms, we first need to understand Convolutional Neural Networks. Convolutional Neural Networks are a computational model that shares certain properties with the 'brain', in which many simple units operate in parallel without a central control unit. The behavior of neural networks is shaped by their network architecture. The architecture of a network can be defined (in part) by the following: Number of neurons, Number of layers, Types of connections between layers. The most well-known and most understandable neural network is the feedforward multi-layer neural network. It has an input layer, one or more hidden layers, and a single output layer. Each layer can have a different number of neurons and each layer is fully connected to the adjacent layer9.

.

9 Josh Patterson & Adam Gibson, 2017, Deep Learning: A Practitioner's Approach, Published by O'Reilly Media Inc.

In the following image, the filtering of an image by the neural networks:

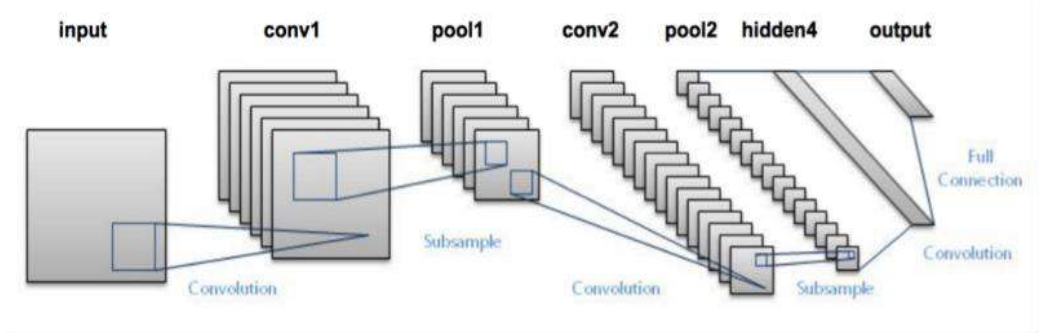

Figure 2.4 Image filtering by convolutional neural networks

A region of interest enters the network and then passes through various convolutions and pooling layers. In the end, we obtain the output in the form of the object's class. For each input image, we obtain a corresponding class as output. We can use this technique to detect various objects in an image.

Convolutional Neural Networks (Convolutional Neural Networks)) which we saw above, is a method of computer image reading, where each pixel of the image is characterized by an atomic number based on color, from 0 (white) to 256 (the darkest shade of green). With this method, the computer understands the sequence of colors and thus can also isolate specific regions to recognize the objects.

For example, in the following image, initially color values of the pixels are taken, while afterwards these numbers (and the ι shades) are multiplied by 3, because it is difficult to discern green with the naked eye without the multiplication. In practice, the multiplication does not occur, however it is done for learning purposes in this case. Thus we observe that there have been created color regions, which the neural networks will subsequently isolate. [10]

10 https://www.analyticsvidhya.com/blog/2017/06/architecture-of-convolutional-neural-networks-simplified-demystified/ (Accessed May 18, 2022)

25	2	1	44			=B2*G5	6	3	132
223	7	6	60			669	21	18	180
196	8	2	148		Weight	588	24	6	444
249	1	3	40		3	747	3	9	120
60	7	1	154			180	21	3	462
59	1	7	213			177	3	21	639
214	7	3	163			642	21	9	489
89	182	219	13			267	546	657	39
74	146	113	72			222	438	339	216
89	18	244	85			267	54	732	255
1	4	8	97			3	12	24	291
3	4	2	121			9	12	6	363
2	1	2	131			6	3	6	393
7	6	8	47			21	18	24	141
3	5	5	126			9	15	15	378
7	6	8	121			21	18	24	363
5	3	1	237			15	9	3	711

Figure 2.5 How convolutional neural networks operate

Simple way of performing Object Detection using Deep Learning

We will see how we can solve with a simple application of the above a general detection problem, so that the way is understood better of operating with Deep Learning using this specific method.

Firstly we will take the following image as input:

Figure 2.6 Example Image

The image is divided into regions and each region is analyzed as a separate image.

Figure 2.7 Image segmentation

Then all these regions (images) are passed through CNNs and are classified into various categories. Once each region is divided into its corresponding class, all these regions can be combined to obtain the original image with the objects detected.

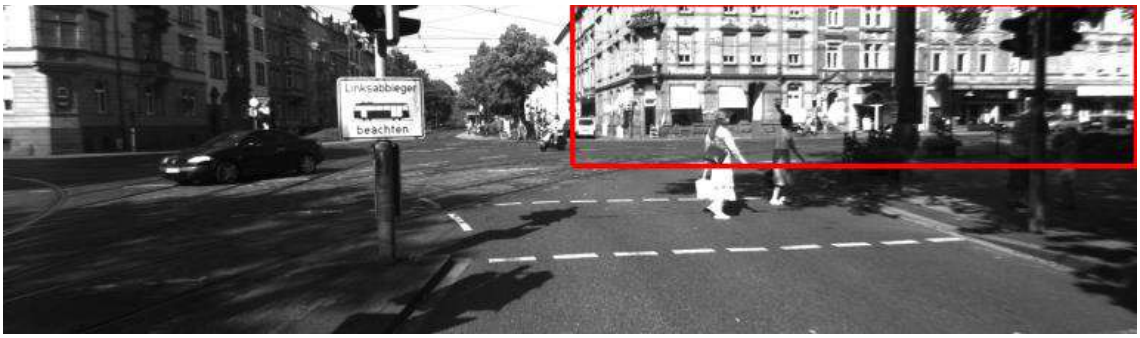

Figure 2.8 Region Categorization

The problem with using this approach is that the objects in the image can have different sizes and spatial positions. For example, in some cases the object may cover a substantial part of the larger part of the image, whereas in other cases the object may cover only a small percentage of the image. The shapes of the objects can also be different. As a result of these factors, we would need a very large number of regions, resulting in a huge computational time. Thus, to solve this problem and reduce the number of regions, we can use a region-based CNN, which selects regions using a proposal method11.

11 https://www.analyticsvidhya.com/blog/2018/10/a-step-by-step-introduction-to-the-basic-object-detection-algorithms-part-1/ (Access May 18, 22)

Single Shot Detectors - Single Shot Detector

Single Shot Detectors (Single Shot Detectors) are based on a set of predefined regions, rather than using a subnetwork to propose regions. A grid of anchor points is placed on the image of input and at each anchor point multiple boxes are created that differ in shape and size, thus defining the regions. For each box at each anchor point, the model outputs a prediction of whether an object exists in the specific region and adjusts the position and size of the box to better fit the object. Since there are many boxes at each anchor point and anchor points can be close to each other, they, SSDs produce many possible detections that overlap. A final post-processing step is applied to the SSD outputs to eliminate most of these predictions and select the best. The most popular post-processing technique is known as non-maximum suppression. .

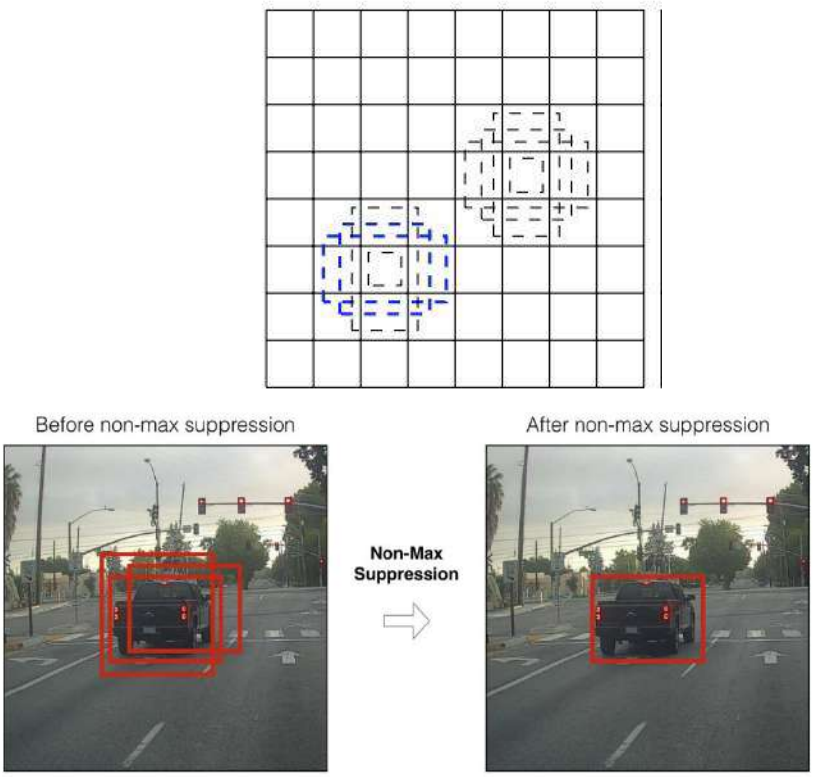

Figure 2.9 Prediction of the most likely anchor points with non-maximum suppression SSD

Region-Based Convolutional Neural Network (Region-Based Convolutional Neural Network)

The R-CNN algorithm proposes a set of bounding boxes in the image and checks whether any of these boxes contain any object, thus avoids working on a large number of regions. The R-CNN uses selective search to extract these region proposals from an image.

First, let's understand what selective search is and how it identifies the different regions. There are basically four regions that make up an object: varying scales, colors, textures and context. The selective search identifies these patterns in the image and based on this, proposes various regions. Here is a short overview of how it works: selective search:

First, we take the input image

Figure 2.10 Example Image

Next, it creates initial sub-subdivisions so that we have multiple regions from this image

Figure 2.11 Image Subdivision

The technique groups similar regions to cluster them and to form a larger region (based on color similarity, texture similarity, size similarity and shape compatibility):

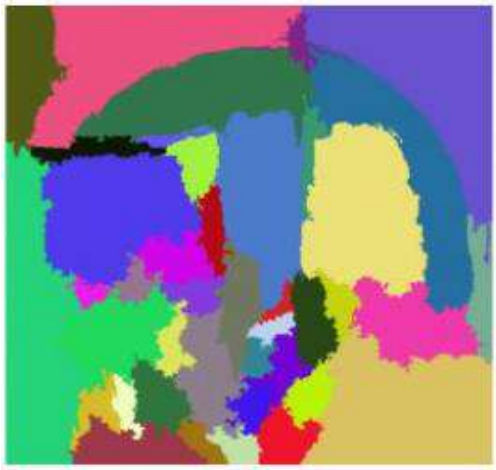

Figure 2.12 Segmentation of Core Regions

Finally, these regions then yield the final object locations of objects.

Now a concise summary of the steps that are followed in R-CNN for object detection in a random image:

Figure 2.13 Introduction Figure

We take the Regions of Interest (ROI) using some proposal method. Then all these regions are reshaped according to the CNN input and each region is fed into the ConvNet:

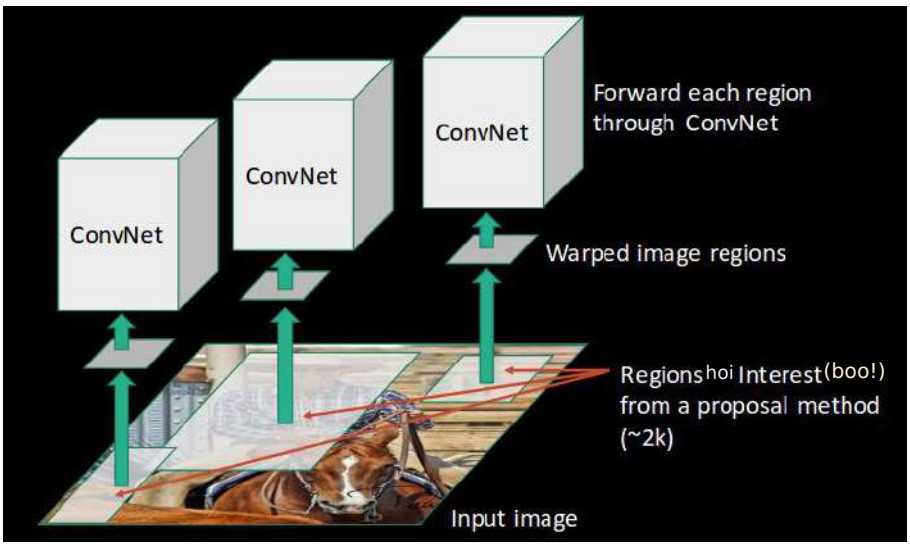

Figure 2.14 Region Processing

Next, the CNN extracts features for each region and the SVMs are used to separate these regions into different categories:

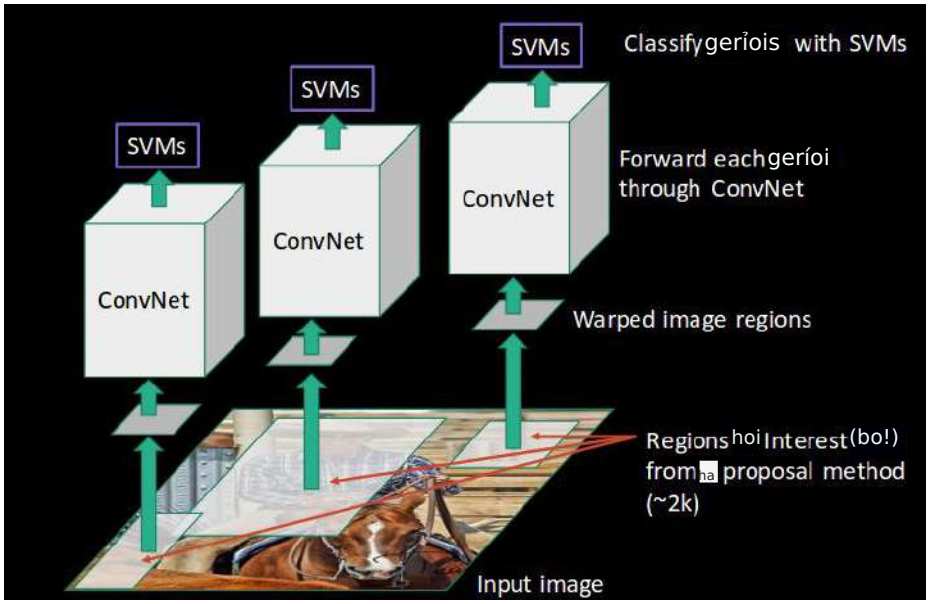

Figure 2.15 CNN Contribution to Regions

Support Vector Machines (SVM) are a set of supervised learning methods used for classification, regression and outlier detection12.

Finally, a bounding box regression (Bbox reg) is used for predicting the annotated boxes for each defined region:

12 Richard Szeliski, 2011, Computer Vision: Algorithms and Applications, Published by Springer London Dordrecht Heidelberg New York

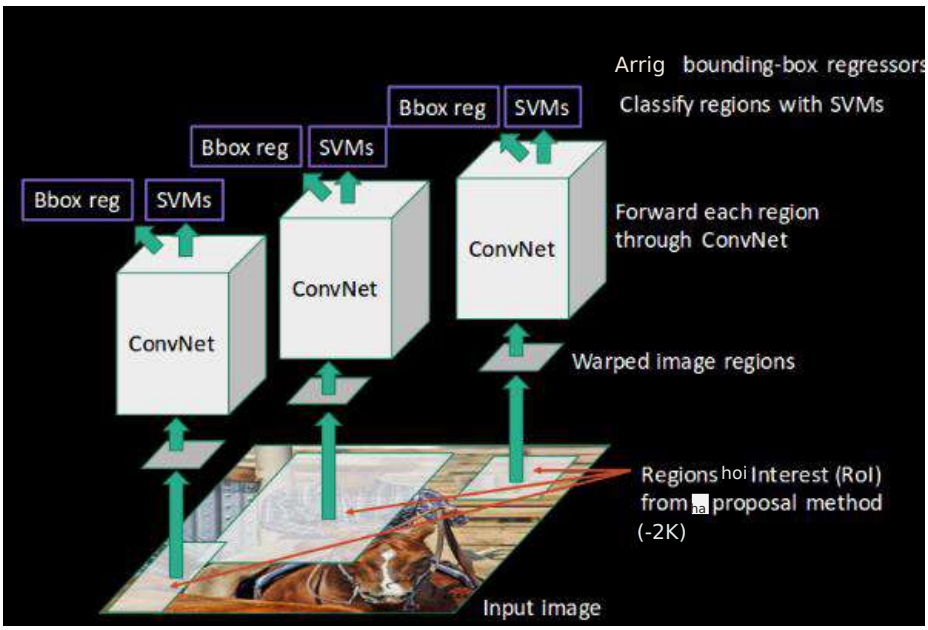

Figure 2.16 Prediction of bounded boxes

Faster R-CNN13

To analyze the Faster R-CNN algorithm we will do a brief overview of the different algorithms in the R-CNN family (R-CNN, Fast R-CNN and Faster R-CNN) for object detection.

The R-CNN we saw above extracts a slew of regions from the region of interest by applying selective search and then checks whether any of these boxes contain an object. Initially we extract these regions and for each region, the CNN is used to extract specific features, which are used for detection of objects.

On the other hand, Fast R-CNN passes the entire image into the ConvNet, which creates regions of interest (instead of propagating the extracted regions from the image). Also, instead of using three different models (as we saw in R-CNN), it uses a single model that extracts features from the regions, classifies them into different classes and returns the bounding boxes. All these steps are performed simultaneously, making

13 https://www.analyticsvidhya.com/blog/2018/11/implementation-faster-r-cnn-python-object-detection/?utm_source=blog&utm_medium=a-step-by-step-introduction-to-the-basic-object-detection-algorithms-part-1 (Access May 18, 22)

faster than R-CNN.

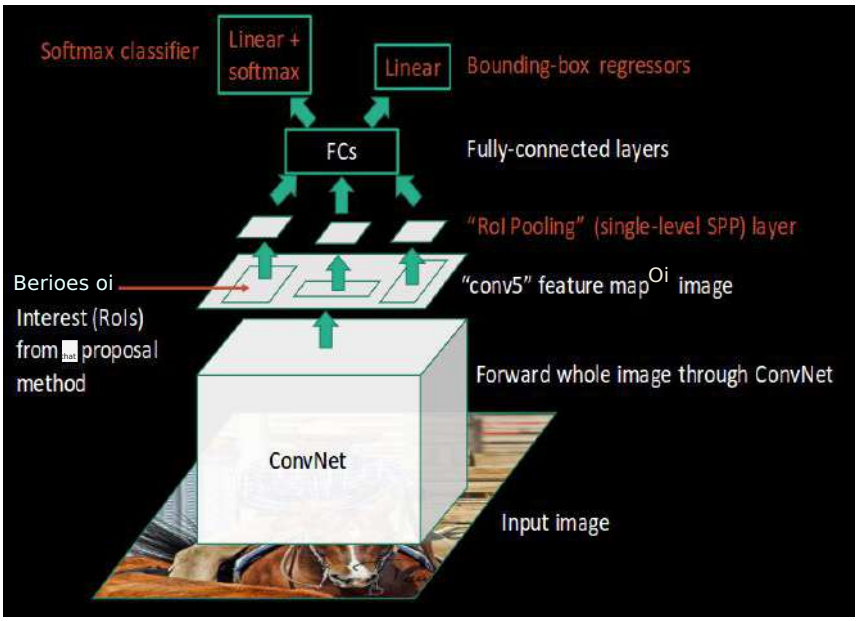

Figure 2.17 How Faster R-CNN works

Faster R-CNN corrects the selective search problem by replacing it with the Region Proposal Network (RPN). First we extract feature maps from the input image using the ConvNet and then pass these maps through an RPN that returns object proposals. Finally, these maps are classified and the bounding boxes predicted

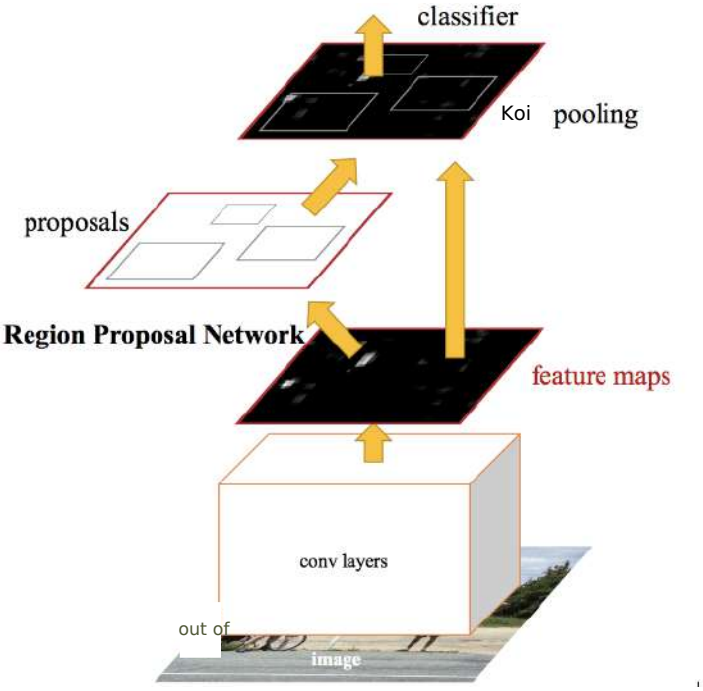

Figure 2.18 Bounding boxes

2.1.4 Comparison of Algorithms

Unfortunately, R-CNN is characterized as slow due to the multiple steps involved in the process. Training an RCNN model is expensive and slow due to extracting ~2,000 regions per image based on selective search and using CNN for each image region. Also the entire object detection process using RCNN has three models: CNN for feature extraction, linear SVM classifier for recognition of objects, a regression model for tightening the bounding boxes.
All these processes are combined to make RCNN very slow.
It takes about 40-50 seconds to make predictions for each new image, which essentially makes the model bulky and practically impossible to build when facing a huge dataset.

However, Fast R-CNN is not fast enough when applied to a large dataset, as it also uses selective search for the extraction of regions.

In Faster-RCNN, object proposal takes time and since different systems operate one after another, the performance of the systems depends on the performance of the preceding system.

In this subchapter, we saw the different models of the R-CNN family and how they differ from each other. We saw how the different region aggregation methods and region proposal methods make changes and also they make the process faster. In modern object detection scenarios, there are some new algorithms such as YOLO and RetinaNet which are also models that maintain speed and accuracy14.

14 https://analyticsindiamag.com/r-cnn-vs-fast-r-cnn-vs-faster-r-cnn-a-comparative-guide/ (Access May 18, 22)

Algorithm	Eeaíyges	Rteaiioiop. time / image	{ipitaíions.
CNN	Oiniaes epé image ipio multiple teríons and depth classify each region into various classes.	tou	Neeás aioi oi gebíons io predict accurately and hence high computation time.
BONN	Uses oiseííno regions. Extracts atoupa 2000 regions Itopi each image.	o... | 0 seconds	uses three different model eog making predictions.
Eaoí BONN	Each image is passed only once to the ONN apa feature maps oge extracted. 5e]eothne search is (56 ou these maps to generate predictions. Combines 8}} the tígee proáeis used in AONN together.	2 seconds	theíesíine 5vâgon es oíonn and hence computation time is still high.
Faster BONN	Replaces the selective search method with region rgorothhi peinnongk which thidas {pe algorithm much [aoioi.	0.2 seconds	Object rtoroea! takes {ipi8 apa a5 pote g' are different s tems workir 8 one aio; {pe other, epe performance oi systems avrepath oi how the rgenioii5 thgthiebi has performed.

Table 2.1 Comparison of Object Detection Algorithms

2.2 INTRODUCTION TO OBJECT TRACKING

2.2.1 What is Object Tracking

The tracking of an object, or otherwise Object Tracking, is a process where the algorithm tracks the movement of one or more objects by estimating or predicting the positions and other relevant information of the moving objects in a video.15

There are two types of Object Tracking:

1) Single Object Tracking (SOT)

2) Multiple Object Tracking (MOT).

15 https://www.v7labs.com/blog/object-tracking-guide#what-is-object-tracking (Accessed 18 May 22)

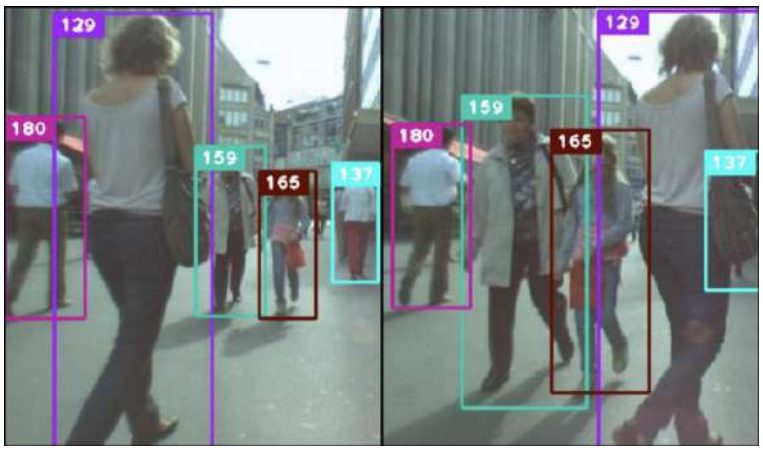

Figure 2.19 Example Object Tracking

Multiple Object Tracking (MOT), also called Multi-Target Tracking (MTT), is a computer vision task that aims at the analysis of video for locating and tracking objects that belong to one or more categories, such as pedestrians, cars, animals and inanimate objects, without knowledge of the appearance and the number of targets.

While in Single Object Tracking (SOT) the appearance of the target is known, in MOT a detection step is necessary to locate the targets that can exit or enter the scene. The main difficulty in simultaneous tracking of multiple targets arises from the various interactions between objects which can sometimes also have similar appearance.

Most of the latest tracking approaches follow the method of 'Tracking-by-Detection', where they first find objects in the scene and then find the corresponding tracklets (position of it in the next frame) of the objects. Today detectors perform exceptionally well and can be scaled to different scene adaptations

2.2.2 How Object Tracking Works

To understand how Object Tracking works, we will present the common steps followed by its algorithm:

1) Object detection, where the algorithm classifies and detects the object creating a bounding box around it.
2) Assignment of a unique identification for each object (ID).

3) Monitoring the detected object as it moves within frames during the storage of the relevant information.

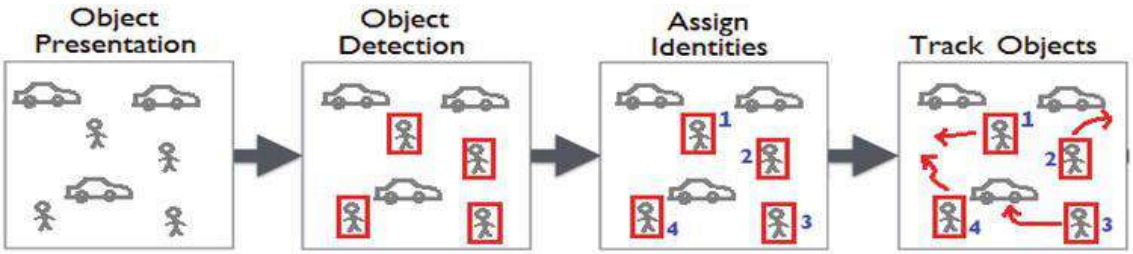

Figure 2.20 Diagram of how object tracking works

Frame difference is the intensity disparity between two frames, assuming that the intensity of a pixel changes, clearly shows something that changes in the image, thus indicating some movement. This is perceived by a simple subtraction of the current image from the previous one or from a reference image. Report. This method has been well studied since the late 1970s, 70s, which forms a foundation for the study of Object Tracking. In general, the method of frame difference is sensitive to lighting changes and cannot detect an object once it becomes stationary. For this reason there is the motion history image, which is obtained with successive layers of frame differences, so that we know at any moment the last recorded position of the moving object before its detection is interrupted for any reason16.

2.2.3 Achievement Algorithms

Convolutional Neural Networks (CNN) remain the most used and reliable network for object tracking. However, several architectures and algorithms are also being explored, among which among which are Recurrent Neural Networks (RNN), Autoencoders Coders (AEs), Generative Adversarial Networks (GAN), Siamese Neural Networks (SNN) and Tailored Neural Networks.

Although Object Detectors can be used for object tracking if applied frame-by-frame, this is a computationally complex method and therefore an inefficient and time-consuming

16 Karman K. and von Brandt A., 1990, Moving object recognition using an adaptive background memory, Published by Elsevier Inc Amsterdam

object tracking method. In contrast, object detection will have to be applied once and then the object detector should be able to handle each frame after the first. Below are concisely presented the most popular Object Tracking algorithms, for a better understanding of operation and the different approaches from the achievement algorithms.[17]

1. OpenCV Object Tracking

OpenCV object tracking is a popular method because it has so many built-in algorithms that are specifically optimized for the needs and goals of object tracking or movement. OpenCV is a C++ library with modules covering many areas of computer vision. Besides C++ (and C) there is growing support for Python as a simpler scripting language through a Python interface on top of the C code base. The Python interface is still under development and not all parts of OpenCV are available and many functionalities are documented. This is likely to change as there is an active community behind it[18].

Some OpenCV Trackers include the trackers BOOSTING, MIL, KCF, CSRT, MedianFlow, TLD, MOSSE and GOTURN. Each of these trackers is better for different targets. For example, CSRT is better when the user requires greater tracking accuracy of object tracking and can tolerate slower FPS performance. The way in which tracker is chosen concerns the advantages and disadvantages of each, in order to have the optimal solution for the requirements of each problem.

2. DeepSORT

DeepSORT is a good choice of tracking algorithm for objects and is one of the most commonly used. The appearance information is embedded in the algorithm, which greatly improves the performance of DeepSORT. Because of the integration, objects can

[17] https://viso.ai/deep-learning/object-tracking/ (Access May 18, 22)
[18] Jan Erik Solem, 2012, Programming Computer Vision with Python, Published by O'Reilly Media

detected in longer periods of obstruction, thereby reducing the number of identity switches.

3. Object tracking in MATLAB

MATLAB is a numerical computation platform, which is different in its implementation compared to DeepSORT and OpenCV, but it is still a good choice for visual tracking tasks. The Computer Vision Toolbox in MATLAB provides tracking algorithms for videos, such as Continuous Adaptive Mean Shift (CAMSshift) and Kanade-Lucas-Tomasi (KLT) for tracking a single object or for use as building blocks in a more complex tracking system.

4. MDNet

MDNet is a fast and accurate optical tracking algorithm based on CNNs, inspired by the object detection network R-CNN. It operates with candidate region sampling and passing them through a CNN. The CNN is typically pretrained on a huge dataset and fine-tuned on the first frame in an input video.

PART II

SYSTEM AND APPLICATION

THIRD CHAPTER

PROPOSED SYSTEM ARCHITECTURE

3 PRESENTATION OF THE PROPOSED ARCHITECTURE

In this chapter the architecture that we chose in order to achieve our goal as quickly and efficiently as possible. A architecture must be built that will serve our problem, while at the same time it will be easily portable and will follow the shortest route.

Below the architecture we arrived at is depicted. Initially We note that the term Area denotes the monitoring space of the program. In this space, we will program our system to automatically record the number of people inside it. The monitoring space can be from a single room to a large building. Depending on our requirements, our architecture is adjusted accordingly. Next we will identify two doors. One is symbolically called the entrance and the other the exit. From these two doors, people will enter and exit, which we will count. It is very important to take into account that from the Entrance door people may go in and out. Similarly from the exit, people may enter. For this reason we have programmed our code to cover all possible contingencies.

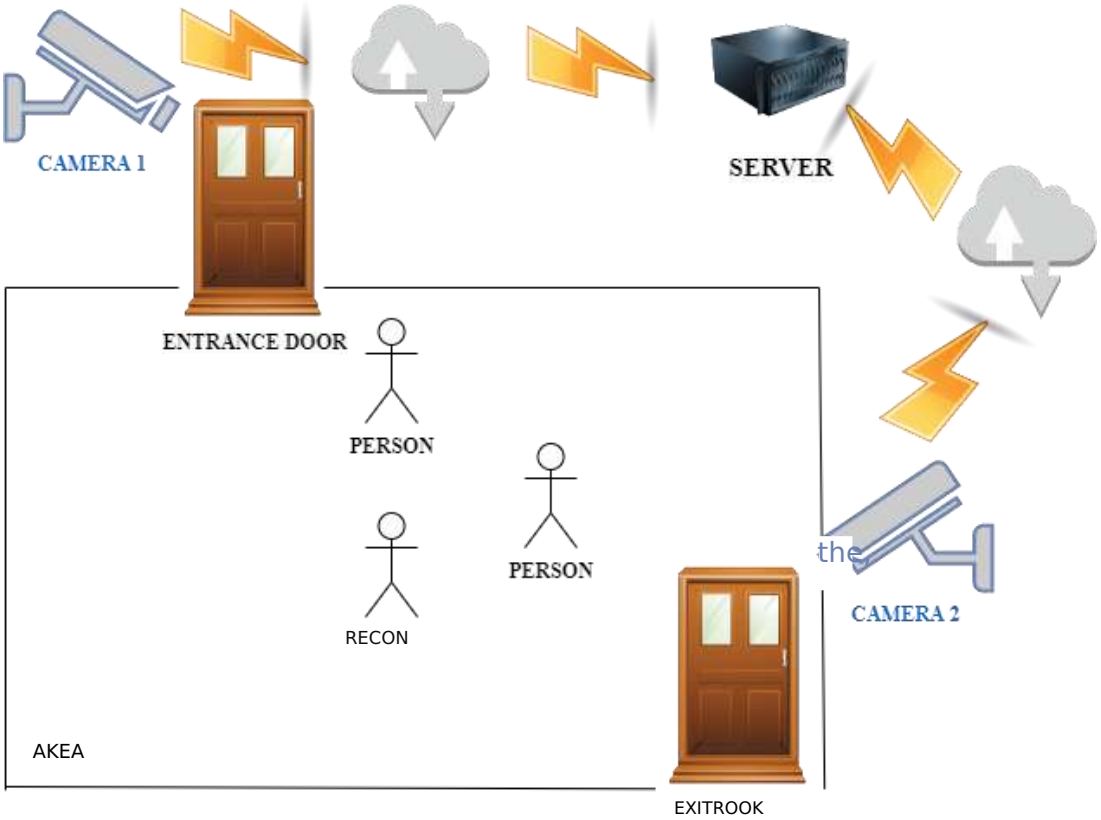

Figure 3.1 Proposed Architecture

Next to the doors we will observe two cameras. Each door will is accompanied by a camera which should not be obstructed by any object and have a good visual field. These cameras monitor the doors and not the space. Each camera has the purpose of counting people for the door it supervises. The supervision of the space will be achieved when the cameras come into communication with each other. Thus, through the communication of the cameras, a 'picture' of our space will be created, and with simple calculations our goal will be achieved. The lightning bolts symbolize communication, and we see that each camera communicates with the Internet, which is symbolized by a cloud. Finally, the two pieces of information from the cameras arrive at the server, where they are taken into account,

and the total count is achieved.

FOURTH CHAPTER

HARDWARE SYSTEM

4.1 EXISTING HARDWARE

4.1.1 IP Cameras

An IP Camera or, alternatively, an Internet Protocol Camera,

An IP Camera or, alternatively, an Internet Protocol Camera, is a type of digital video camera that captures control data and transmits image data over an IP network (Internet Protocol)19. They are generally used for surveillance or monitoring, as in our case. Unlike traditional closed-circuit television (CCTV) cameras, they do not require a local recording device, only a local network. Most IP cameras are webcam-style cameras, but the term IP camera usually applies to those where direct access via network connection is possible, which is typically used for monitoring. It is therefore important to select an IP camera for monitoring a space, since communication between two or more cameras is required. This communication will be achieved through wireless or wired networks20. ε

Figure 4.1 IP Camera

How do IP cameras work? ;

Analog security cameras require a coaxial cable, the which will transmit the footage to a digital video recording device (DVR). In the case of an IP security camera, it can transmit video via wireless

(19 https://en.wikipedia.org/wiki/IP_camera)

20.Vlado Damjanovski, 2014, CCTV: From Light to Pixels (Third Edition), Published Elsevier Inc. by

connection. Specifically, IP cameras connect to a network video recorder (NVR) via Wi-Fi, Ethernet cable, or USB.21

IP cameras record footage in high definition, the resolution of which can reach up to 16 megapixels, depending on the camera model. Each IP camera is equipped with a processing chip that compresses the video as it is recorded. The higher the camera resolution, the more data each video recording contains. High-resolution images require more storage space and more bandwidth for data transmission than lower quality images. To transmit HD images over a network, IP cameras must compress the files or make the files smaller to avoid excessive bandwidth consumption.

To achieve fast data transmission between the camera and server, there are two alternative network options. A wireless network or Wi-Fi network sends and receives data to a wireless modem. A wired network connects an IP camera to the network via an Ethernet cable. This setup is considered the most secure, as there is little chance of signal interference or unauthorized access. In this way, we achieve higher data transmission speeds with Ethernet, as the wired connection is much more efficient and reliable than Wi-Fi.

In our application, at least 2 IP cameras will be needed, which will be connected directly to a computer via cable. Depending on the camera, the connection may differ in type, Ethernet / USB. The IP cameras should have satisfactory resolution in order to maximize the effectiveness of our program.22

.

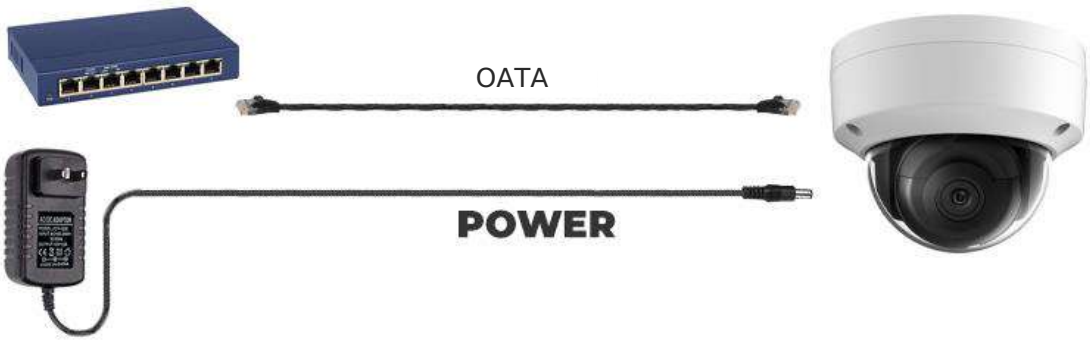

Figure 4.2 Illustration of wired IP camera connection

21 https://www.bayalarm.com/commercial/security-cameras-commercial/what-are-ip-cameras-and-how-do-they-work/#

22 Herman Kruegle, 2007, CCTV Surveillance: Analog and Digital Video Practices and Techonology (Second Edition), Published by Elsevier Inc.

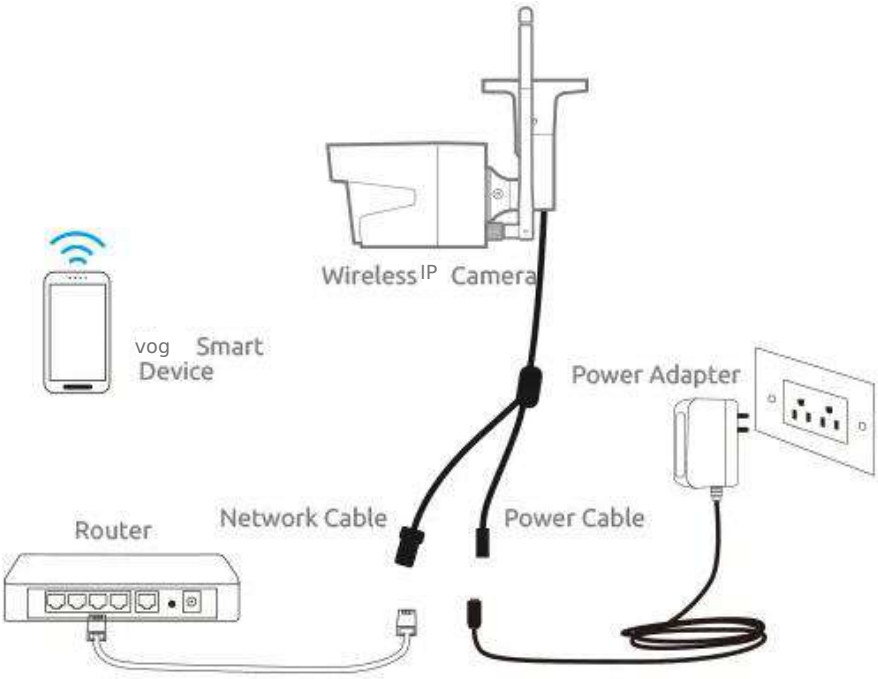

Figure 4.3 Depiction of wireless IP camera connection

4.1.2 Server

In information technology, a server, or alternatively a server, is a piece of hardware or software that provides functionality for other programs or devices, called customers. This architecture is known as a client-server model. Servers can provide various functions, often called services, such as sharing data or resources among many clients or performing computations for a client. A single server can serve many clients and a single client can use many servers. A client process may run on the same device or may connect over a network to a server on a different device23.

23 https://en.wikipedia.org/wiki/Server_(computing)

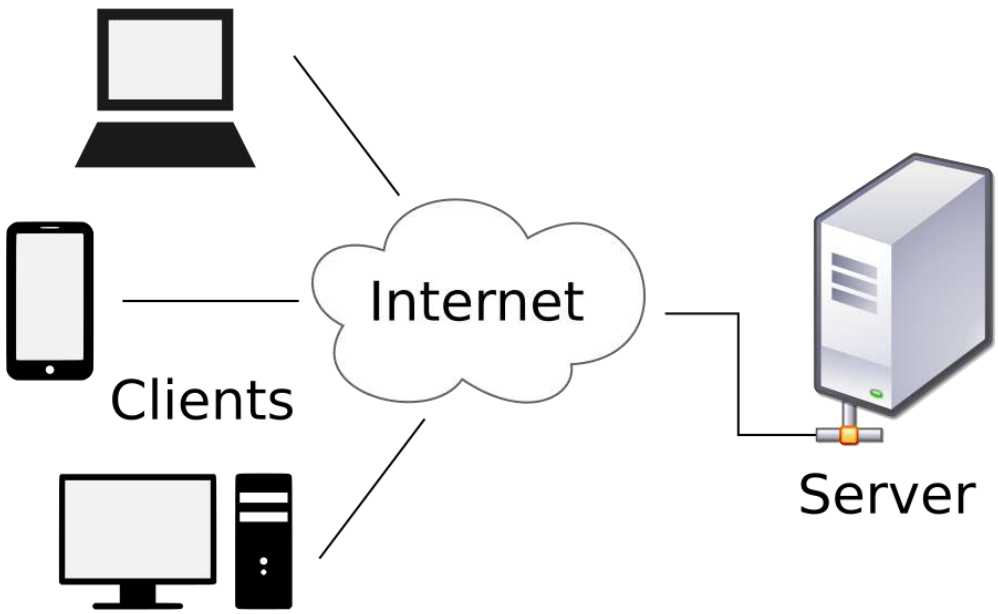

Figure 4.4 Illustration of client-server architecture

Client-server systems are more commonly used in comparison to other systems. They are often identified with the request-response model: a client sends a request to the server, which performs some action and sends a response back to the client, usually with a result or confirmation. The designation of a computer as "server-class hardware" means that it is specialized for running servers on it. This often implies that it is more powerful and reliable than typical personal computers, but alternatively, large computer clusters may consist of many relatively simple, interchangeable server components.[24]

We will now see what a server offers, as opposed to a typical desktop computer. Computers can run server processes, but they are less productive. In contrast, servers offer the following capabilities to the networks they serve:

Scalability to serve an increasing or fluctuating number of devices, users, and workloads.

High processing power with increasing CPU and RAM specifications to handle the network workload.

Reliability to ensure that critical systems remain online and available.

Collaboration among staff with access to shared network resources.

Cost savings over time, because servers can reduce the workload on network devices.

24 https://www.serverwatch.com/guides/what-is-a-server/

To operate a device as a server, it must be configured so that it listens to requests from clients in a network connection.. When a client requests data or operations from a server, it sends a request over the network. The server receives this request and responds with the appropriate information. This is the client–server networking request–response model, also known as the call-and-response model. A server often performs many additional tasks as part of a single request and response, including verifying the identity of the requester, ensuring that the client is authorized to access the data or resources requested, and the proper formatting or return of the required response in an expected manner.

To achieve our application, our server should communicates with at least 2 IP cameras, which we analyzed in subsection 4.1.1. The server we use must meet the minimum characteristics, which are not particularly high. Specifically, our server is characterized by the AMD E2-9000e processor RADEON R2, 4 COMPUTE CORES 2C+2G 1.50GHz, RAM 4 GB, storage 500 GB and Linux Ubuntu software. These are sufficient for the program to run satisfactorily in terms of performance and the speed we require.

For the implementation of the program, besides the electronic computer, which does not need to have particularly high specifications, will we need a mouse, two cameras (in our application we used as) the second camera the camera from the laptop, but if we want with a modification to the parameters we can add a secondary camera, increasing the cost and the hardware of course). Next, there will be a visualization of the existing system and a cost table. The costing has taken into account the minimal system requirements, in order to have a satisfactory result.

25 https://www.paessler.com/it-explained/server

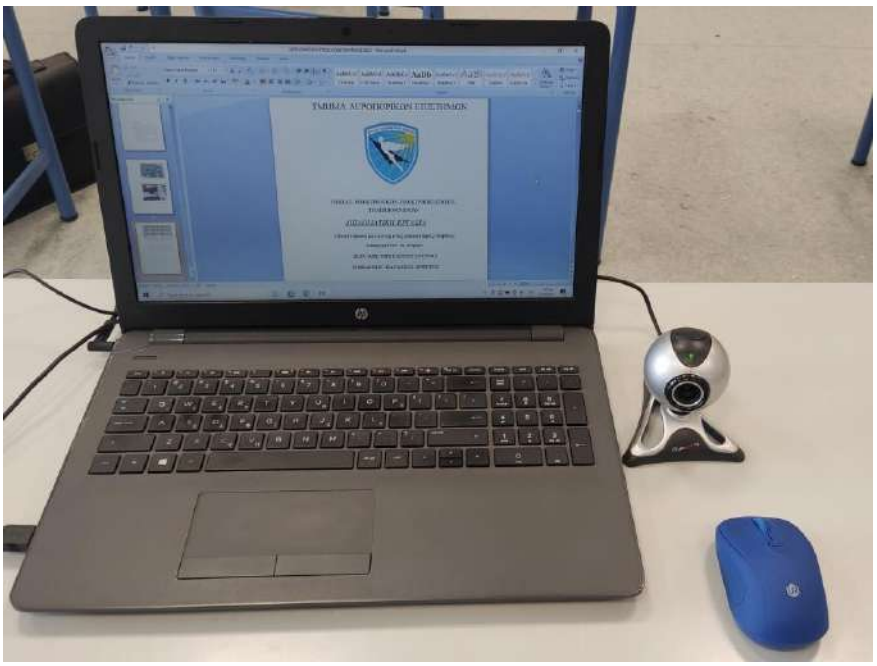

Figure 4.5 Final Operating System

Hardware	Quantity	Cost
Computer	1	minimum 300
Mouse	1	15
Camera	1 or 2	15 or 30

Table 4.1 Consolidated Hardware-Cost Table with the Electronic Method Computer

4.2 RECOMMENDED EXTENSION

4.2.1 Raspberry Pi

By 2006, at the University of Cambridge, a group of professors at the

In 2006, at the Cambridge University Computer Laboratory, a group of professors was concerned about the reduced interest of students in computer science, as well as their limited knowledge. This group, which included Eben Upton, Rob Mullins, Jack Lang, and Alan Mycroft, thought that the solution would be an extremely small and affordable computer. With such an economical system, computer science could be practically taught in schools and attract more interest from students. While the group designed several initial Raspberry Pi prototypes, we must consider that in 2006 there were significant limitations due to the high cost and low power of processors for mobile devices. Gradually, with the release of the first iPhone in 2007 and the subsequent surge of smartphones, the cost of technology began to fall enough to make the Raspberry Pi implementation viable. In 2008, with the collaboration of Pete Lomas and David Braben, the charitable Raspberry Pi Foundation was established.

Three years later, the first Raspberry Pi was released, with Model A, Model A+ and Model B26.

Figure 4.6 Raspberry Pi Logo

So, the Raspberry Pi is a small single-board computer (SBC)) which was developed in 2008 in the United Kingdom by the Raspberry Pi Foundation in collaboration with Broadcom27. Simply put, it is a low-cost computer the size of a credit card that connects to a computer monitor or TV and uses a standard keyboard and mouse. It is a capable little device that enables people of all ages to explore computers and learn how to program in languages such as Scratch and Python. It can do everything you would expect a desktop computer to do, from browsing the Internet and playing high-definition video, to creating spreadsheets, word processing, and playing games28.

.

The Raspberry Pi project initially leaned towards promoting the teaching of basic computer science in schools and in developing countries. Today, it is widely used in many fields, due to its low cost, modularity, and open design.

In more detail, the Raspberry Pi is a fully equipped and functional computer that occupies only the size of a credit card. Despite its minimal size, the Raspberry in one of the

26 https://www.pcsteps.gr/52828-%CF%84%CE%B9-%CE%B5%CE%AF%CE%BD%CE%B1%CE%B9-%CF%84%CE%BF-raspberry-pi/

27 https://en.wikipedia.org/wiki/Raspberry_Pi

28 https://www.raspberrypi.org/help/what-%20is-a-raspberry-pi/

larger and more recent editions (Raspberry Pi 4 Model B) features a quad-core processor at 1200MHz, dual-core graphics card, 1GB RAM, four USB ports, HDMI output, powered via Micro USB, and 40 general-purpose pins for connection with other electronics and peripherals29.

It is a computer that runs the Linux operating system.

It has USB sockets where we can connect a keyboard and mouse and an HDMI video output (High-Definition Multimedia Interface) where we can connect a television or monitor30.

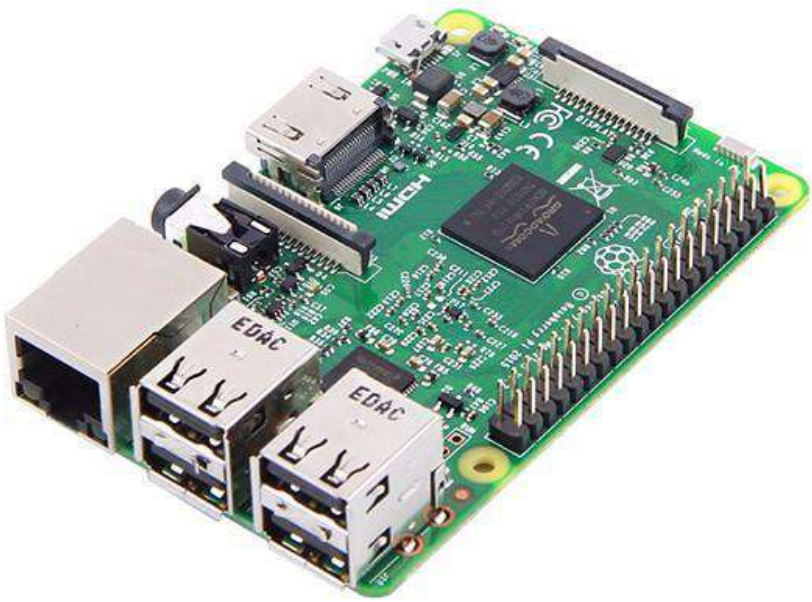

Figure 4.7 Representation of the Raspberry Pi 3

While it is impressive how this high computing power when this large computing power is concentrated in such a small space and at such a low cost, the question arises: what can we do with the Raspberry Pi? What are its capabilities and uses? As it turns out, the programmer with the right imagination, innovation, and effort makes Raspberry applications practically limitless. Some standout applications are the following:

Education: The Raspberry contributes significantly to educational purposes, since the initial idea that led to its construction was an affordable, simple and small computing system that could be used by everyone.

29 Sean McManus and Mike Cook, 2013, Raspberry Pi for Dummies, A Wiley Brand

30 Simon Monk, 2013, Programming the Raspberry Pi: Getting Started with Python, Published by McGraw-Hill Companies

Home automation: An extension of its applications is that

home automation, which was created either by private developers to solve everyday problems or by companies that manufactured

programs that simplify and digitalize daily life at home.

Industrial automation: Its contribution is also great

Raspberry in the industrial sector, as it is an economical solution for automating various functions of companies, etc. A major advantage is also the small space it occupies, since for example in a factory the automated systems required can number in the hundreds to thousands. It is also a significant pillar the small size of such a system when thousands of them are required.

Covid-19 Pandemic: The pandemic we are all experiencing brought new

needs and a new daily life. The contribution of Raspberry to these changes is enormous, as automation of various tasks, monitoring and surveillance of closed spaces and other applications has managed to reduce congestion, make our lives easier, and prevent, this technological means, to a large extent, the spread of it.

4.2.2 Extension proposal using Raspberry

Our final system (existing system) differs from the

The final system we present here (existing system) differs from the one proposed in this subsection. The reason it differs is due to various factors. So the proposed system is based on developing our program on Raspberry 4. The Raspberry prevails over the computer for various reasons. Specifically, the Raspberry occupies less space and is more portable, so despite its somewhat higher cost, it would be a better choice, for example for a company that would need a heavier and more demanding system with more than two cameras.

During our work, we developed the system with

Raspberry Pi 3. However, due to the system's inability to meet the requirements, we chose to work on a computer. More specifically, the Raspberry Pi 3 did not have enough processing power, so when the program was completed and run from the Raspberry, it could not run satisfactorily, it ran at a rather slow rate and not very efficiently.

The system we propose consists of Raspberry Pi 4 as a

server, compared to Raspberry Pi 3, which has better performance and greater power. This system will run much more efficiently and at a better fps (frames per second) rate.

And the hardware report we propose to use,

is based on the tested system, an effort that was made using two

Raspberry Pi 3. This system, despite its failure to support the demanding

our program, is important to also calculate its minimum cost so that

set up. The cost of such a system with all components is calculated to be will require a calculable amount, which makes us wonder whether it is worth in the final analysis this investment, when a simple laptop does exactly the same job and with greater success.

Below is shown in figure 4.7 the Raspberry Pi 3, while in figure 4.8 the complete system. An analytical table of the hardware we used and the calculable cost of it.

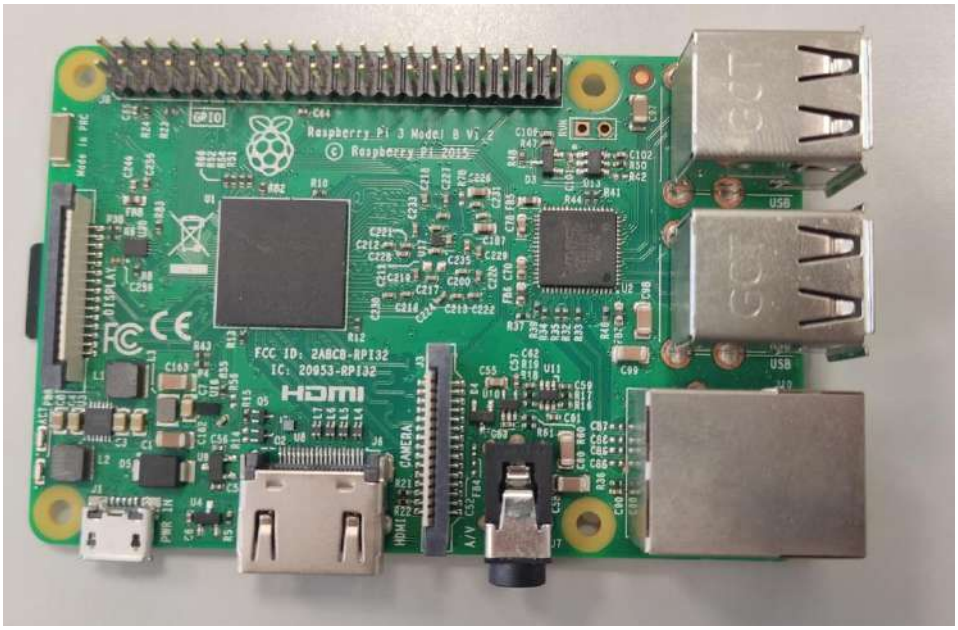

Figure 4.8 Raspberry Pi 3

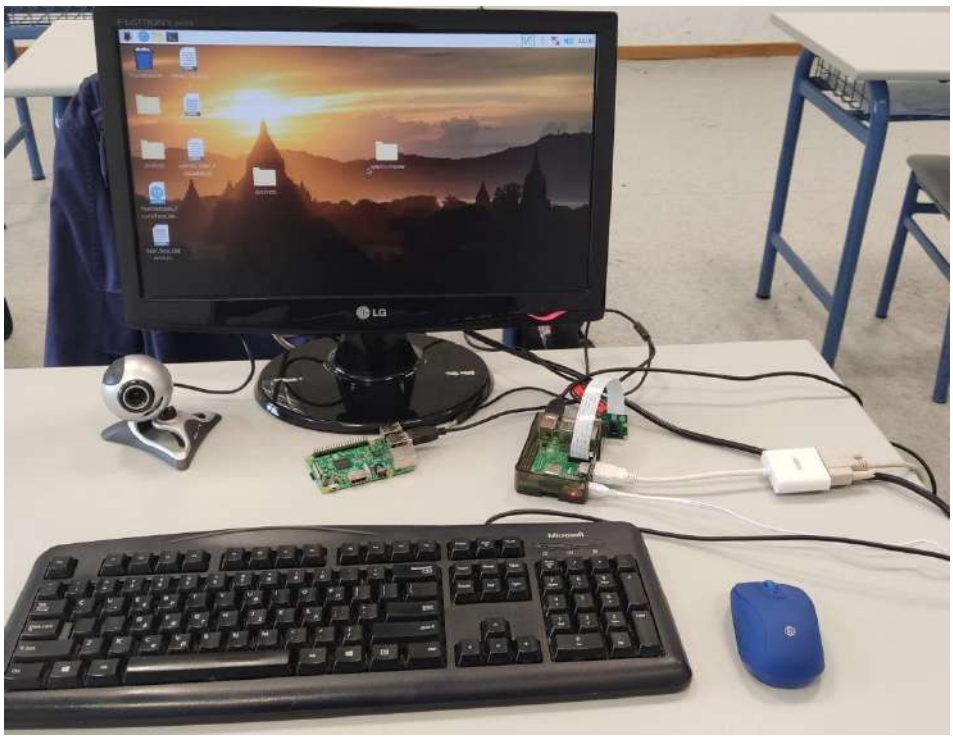

Figure 4.9 Integrated System using Raspberry

Hardware	Quantity	Cost
Raspberry Pi 3	2	120
Raspberry Cameras	2	100
Monitor	1	60
Mouse	1	15
Keyboard	1	15
16 Gb Memory Cards	2	30
Raspberry Power Supply	1	10
Cameras	2	30
Other (cables, power strip, adapters)	1	~10

Table 4.2 Summary Table of Hardware-Cost using the Raspberry Pi method

FIFTH CHAPTER

SOFTWARE SYSTEM

5.1 INTRODUCTION TO DEEP LEARNING

For the design, construction and application of our program, we will use Deep Learning as a basis, which will have as main tools the Object Detection and Object Tracking, which we saw in Chapter 1.

Deep Learning is a category of machine learning algorithms and artificial intelligence (AI) that mimics how humans acquire certain types of knowledge. It deals with algorithms inspired by the structure and function of the brain known as artificial neural networks. The word "Deep" refers to the use of multiple levels in the network, as it deals with an unlimited number of layers of limited size. It uses multiple levels to gradually extract higher-level features from the raw input. For example, in image processing, the lower levels can define the boundaries of an object, while the higher levels can define concepts such as digits or letters or faces.

Deep Learning is an important element of data science, which includes statistical and predictive models. It is extremely beneficial for data scientists who are charged with collecting, analyzing and interpreting large volumes of data. Deep Learning makes this process faster and easier.

In its simplest form, it can be regarded as a way to automate predictive analytics. While traditional machine learning algorithms are linear, deep learning algorithms are stacked in a hierarchy of increasing complexity and abstraction.

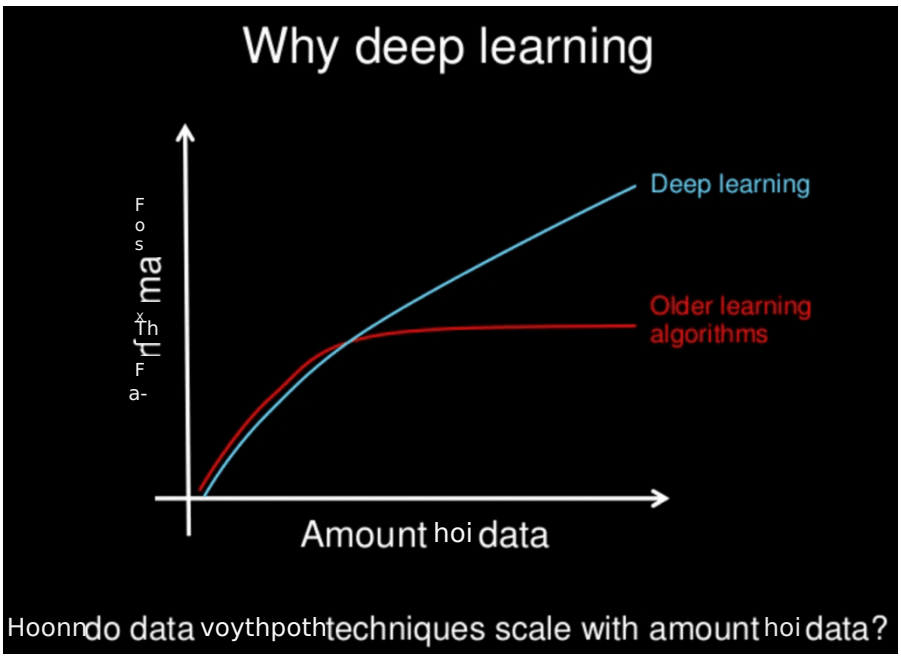

Figure 5.1 Comparison of Traditional Algorithms with Deep Learning

How does it work?

Computer programs that use deep learning go through the same process until it learns to recognize the object. Each algorithm in the hierarchy applies a nonlinear transformation to its input and uses what it learns to create a statistical model as output. Iterations continue until the output reaches an acceptable level of accuracy. The number of processing levels the data must pass through is what inspired the label 'deep'.

In traditional machine learning, the learning process is supervised, and the programmer must be extremely specific when telling the computer what kind of things to search for to decide if an image contains a particular object or not. It is a tedious process called feature extraction, and its success rate depends entirely on the programmer's ability to define with precision a set of features for the object. The advantage of deep learning is that the program creates the feature set by itself without supervision. Unsupervised learning is not only faster, but it is usually more accurate.

Initially, the computer program can be supplied with data for training -- a set of images for which a human has labeled, for example, each image as dog or not dog. The program uses the information it receives from the training data to create a set of features for the dog and to build a predictive model. In this case, the model that the computer first creates may predict that anything in an image with four legs and a tail should be classified as a dog. Of course, the program does not know the labels four legs or tail. It will simply look for pixel patterns in the digital data. With each iteration, the prediction model becomes more complex and more accurate.

Unlike a 'toddler', who would need weeks or even months to understand the concept of a dog, in a computer program that uses ‘deep learning’ algorithms, you can show it a training set and it can classify millions of images, accurately recognizing which images have dogs in just a few minutes.

Because the process of training a model is time-consuming and complex, we will use pre-trained models, which recognize the human form and movement.

5.2 REQUIRED SOFTWARE

For the construction of the program, it is necessary to use software that does not significantly burden the computer. The complexity of the program, its calculations and in combination with the use of Deep Learning, make it harder for the computer and we would not want to burden it further with any heavier software.

The program was initially designed and tested on a Raspberry Pi 3, but despite efforts and addressing various problems that were identified along the way, it turned out that this particular computer was unable to support such a demanding application. As a result, the program ran very slowly and failed to detect all people, either

they entered or exited the space. Note that the Raspberry runs with Raspberry Pi OS software or Rasbian as it used to be called.

Since the attempts to set up our program on the Raspberry failed, we tested its application on a simple laptop, with which our application ran quickly (at least 10 times faster) and efficiently.

For the computer, we wrote the program in the programming language Python, as in the Raspberry too. Python is a relatively easy language of programming which finds application in schools and universities due to its simple structure and syntax. This particular programming language significantly facilitates the programmer's work while at the same time does not require any special computer hardware to support it.

The reason we chose Python is because of its simplicity, its straightforward structure, and its speed. It is powerful, yet very accessible. Python has become the most popular programming language for data science, because it lets us forget the tedious parts of programming and provides an environment where we can quickly annotate our ideas and turn concepts into action.

.

The purpose of the program we build is simplicity and effectiveness, without requiring an expensive computing system to support it.

The programming language we mentioned above, which we, implemented with the help of Thonny. Thonny is an integrated development environment for Python designed for beginners.

The program we will build requires some specific inputs and tools, which are called 'libraries', i.e., libraries.

What is a Python library?

A Python library is a collection of related functional units. Contains bundles of code that can be reused across different programs. As a result, Python programming becomes simpler and more convenient for the programmer.[32] Following is a numbering of the libraries and a brief description of their functionality.

The essential libraries we installed and used are:

OpenCV

NumPy

---

31 Sebastian Raschka & Vahif Mirjalili, 2019, Python Machine Learning, Published by Packt Publishing Ltd.

32 https://www.geeksforgeeks.org/libraries-in-python/

Imutils

dlib

OpenCV (Open Source Computer Vision Library) is a library of open-source computer vision and machine learning software.

NumPy is a library for the Python programming language that offers support for simple as well as large multidimensional arrays, along with a large collection of high-level mathematical functions for operating on these arrays.

Imutils is a collection of convenience functions that simplify basic image processing operations, such as translation, rotation, resizing, skewing and displaying images.

Dlib is a general-purpose, multi-platform software library written in C++. Therefore, it is, first of all, and mainly a set of independent software components. It provides great help in face detection with pretrained models that are used to estimate its[33] coordinates.

5.3 REQUIRED WORKING ENVIRONMENT

A brief report about the prerequisite will follow. the environment necessary for the system to run.

Initially, the operating system we will use is Linux. In the Linux, after updating the system via the terminal and the commands $sudo apt-get update and $sudo apt-get upgrade, the installations of NumPy, OpenCV, dlib, and imutils[34] must be performed.

The installation of OpenCV[35] is a bit more complex and time consuming. We install two essential packages with the commands:

$ sudo apt-get install python3-dev

$ sudo apt-get install libgl1-mesa-glx

We should also install pip: :

33 Zed Shaw, 2014, Learn Python the Hard Way (Third Edition), Published by Addison-Wesley
34 David Beazley & Brian K. Jones, 2013, Python Cookbook, Published by O'Reilly Media Inc.
35 OpenCV Installation: https://pyimagesearch.com/2018/09/19/pip-install-opencv/

$ wget https://bootstrap.pypa.io/get-pip.py

$ sudo python3 get-pip.py

Next, we install the following, which will help us via pip to create a virtual environment:

$ pip install virtualenv virtualenvwrapper

We create a virtual environment named cv:

$ mkvirtualenv cv -p python3

And now we simply install OpenCV via pip:

$ pip install opencv-contrib-python

Next is the installation of dlib[36], NumPy and imutils which starts with the installation of necessary packages:

$ sudo apt-get install build-essential cmake

$ sudo apt-get install libopenblas-dev liblapack-dev

$ sudo apt-get install libx11-dev libgtk-3-dev

$ sudo apt-get install python python-dev python-pip

$ sudo apt-get install python3 python3-dev python3-pip

Next, with the command $workon cv, we enter the virtual environment that we created and simply run $pip install dlib, $pip install numpy and $pip install imutils.

[36] Installation of dlib, numpy, imutils: https://pyimagesearch.com/2018/01/22/install-dlib-easy-complete-guide/

SIXTH CHAPTER

SYSTEM PRESENTATION

6.1 SYSTEM CODE ANALYSIS

Next, we will present the code and analyze it

in sections. The code contains comments which are introduced with the symbol #..

Opening the code People_Counter.py, we initially observe that we import the necessary packages with the import command. From the pyimagesearch module, we import the custom classes CentroidTracker and TrackableObject. VideoStream and FPS will help us calculate the FPS rate of the program. The imutils import offers convenience for the demanding function calculations that OpenCV will perform. The dlib library will be used for the program's tracking. Finally, OpenCV will be used for extracting conclusions in a deep neural network and displaying output frames on our screen.

```
1  # tk  IV VIZOTH KONTHTANTINOS  : Rgorcdpi {Pan  automatically oupis   people inside an area
2  # Eigthx na iproge epe    necessary packages
3  Egopi  pyimagesearch.centroidtracker import CentroidTracker
4  Egopi  pyimagesearch.trackableobject import Trackabl THO ject
5  Egopi  imutils.video import VideoStream
6  Egopi imutils.video aproge ER5
7  from po] tiprocessing eprogx  Manager
8  Egopi pia] tiprocessing import Process
9  aprognx pyprn 85 pr
10 import argparse
11 aprognx Ipmis
12 aprognx Tpe
13 import a110
14 import on2
15 import os
16 apirogx  sys
```

Figure 6.1 Code Snippet of the Program

Next, we will import the list that contains the elements that the

pre-trained MobileNet SSD model has learned to recognize. We remind

that the process of object detection is time-consuming and complex, therefore

it is not necessary to build it since there are already

pre-trained models. In this way, the programmer's job is made easier

programming. Also, with the args command, we provide the command line parameters

The command line expects the user to provide 6 parameters which

are: :

--prototxt : Path to the prototxt "deploy" file of Caffe

--model : The path to the pre-trained Caffe model.

--confidence : With a default value of 0.4, this is the minimum threshold of probability that helps filter out weak detections.

--skip-frames : The number of frames to be skipped before the detector is run again on the tracked object.

We also define and set the Grand_total parameter to zero. In lines 33 and 34, we load the pre-trained model for each camera MobileNet SSD, which is used to detect objects.

```
18 or Insert the list not capablelabels that MobileNet SSD was trained the detect
19 CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat",
20     "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
21     "dog", "horse", "motorbike", "person", "pottedplant", "sheep",
22     "sofa", "train", "tvmonitor"]
23
24 which to construct {pe t parse and parse the arguments Tos each camera
25 args = , prototxt :"podl]epei ssd/MobileNetSSD_deploy.prototxt", "model" : "mobilenet_ssd/MobileNetSSD_deploy.caffemodel"
26         ,"prototxt1" . et_ssd1/MobileNetSSD_deploy.prototxt", "model1" : "mobilenet_ssd1/MobileNetSSD_deploy.caffemodel",
27         "confidence" : 0.4 , "skip-frames" : 30 }
28
29 # with set Grand_total equal io zero 0
30 Grand_total=0
31
32 or loda on2, serialized model egop disk two times , [or each camera
33 pei = dnn.readNetFromCaffe(args["prototxt"], args["model"])
34 net1 dnn.readNetFromCaffe(args["prototxt1"], args["model1"])
35
```

Figure 6.2 Program Code Snippet

Next, we activate the 2 cameras which have been set as scr=0 and scr=2. We set W and H as the width and height of the window that will appear during the execution of the program. We also create the centroid tracker. .

We have ct: our CentroidTracker, trackers: A list for storing the dlib correlation trackers, trackableObjects: A dictionary that maps an object ID to a TrackableObject. The initialization of these variables happens twice, once for each camera.

```
# oreē camera 1 ap camera 2
print("[INFO] starting Eperapse thoon Camera ...")
ns - VideoStream(src=0).start()
time.sleep(2.0)
print("[INFO] starting Exit Door Camera...")
nai - VideoStream(src=2).start()
time.sleep(2.0)

andinitialize the egdpth dimensions
W - Nope
Or - Nope

theinstantiate oug centroid egvthokog,then Ip1]Tha]IlZo a 1151 To sioge
and each where our 4110 correlation trackers, To]]onra on a aisiopagn to
# map dthsi upyiaue object TO To a TrackableObject
ei - CentroidTracker(maxDisappeared=40, maxDistance=50)
Egraskors -[ ]
trackableObjects - {}

ct1 CentroidTracker(maxDisappeared=40, maxDistance=50)
trackers1 - []
trackableObjects1 - {}
```

Figure 6.3 Program Code Snippet

Next, we initialize the following variables:

totalFrames: The total number of frames that were processed for processing.

totalDown and totalUp: The total number of objects/people that have moved either downwards or upwards.

fps: The estimator of frames per second.

```
60 # initialize the total number ou Eraspos processed epys far, along
61 i with ape total number ou objects epde have pontha ei ther up or aonp
62 totalFrames - 0
63 toid]thoni - 0
64 totalUp - 0
in 6' toia]thomhipi - 0
d 6! totalUp1 - 0
67
68 # start the Ericapos per second to Gouveryx estimator
6 [Ho Ers - FPS().start Ho)
70
```

Figure 6.4 Excerpt from Program Code

An iterative loop on line 72 starts and we set from where it will reads the each pop-up window the image that will be displayed on the lines 75-76. Next, we set the width of the windows while converting to RGB

our image in lines 81-85. We set values at the frame boundaries if not provided in lines 89-92.

```
71  From 1oor one You wrote against you not now stream
72  while True:
73              # grab before pechú egyde and Papa]e 1 me  are ge8aipr you wrote to be ther
74              or niaeozapιying on VideoStream
75          - Written - vs.ge8a()
76          frame1 vs1.gthda()
77
78              x ge8sizth the Egape To have ha pdchipip width ou 700 riches
79              Ex then convert
80              or the Wrote egopi BWA To Kath Ton a115
81          egapth - imutils.resize(frame, width=700)
82          rgb cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
83
84          frame1 imutils.resize(frame1, width=700)
85          GROI - cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)
86
87
88              E 1E the Eggrp s dimensions 8ga empty, 5ei them
89          if W 15 Nov in H 35 Nov:
90                  (H, N) - egthpθ. shape[:2]
91          ΔEN 15 Νοπε ον Η 35 Νοπε:
92                  (H, N) - frame1.shape[:2]
```

Figure 6.5 Program Code Snippet

Subsequently, on lines 97-100 we set the state ‘Waiting’. In the waiting state Waiting the program waits to detect a target-person. Below the state ‘Detecting’ is defined, which is activated when through MobileNet SSD we detect a person. There is also a state, the 'Tracking' state which is activated when people have been detected in the frames and the program counts totalUp and totalDown. .

To avoid running the object detector on every frame and to speed up tracking, we will skip every N frames (which is defined by the command line argument --skip-frames where 30 is the default). Only every N frames will we perform object detection. The interval is very small and does not affect the performance of the program. Assuming we have detection, we will update the state to "Detecting" (Line 107,109). Next, we initialize our new list of trackers (Line 108,110). Then, we will perform inference through object detection. We start by creating a snapshot from the image, passing the snapshot through the net to get detection (Lines 114-120).

```
94          Theinitialize the current status 8]opb with our list who bounding
95          # box rectangles Gezurptha og either (1) our ou]thsi detector or
96          8 (2) - the correlation trackers
97      status "Waiting"
98      gesis - []
99      status1 - "Naleir"
100     rects1 - []
101
102
103         # check io sos 1E me should gyp 8 more computationally expensive
104         4 object detection method heo aid oug tracker
105     1E  totalFrames The 30 -- 0
106                 or set epe sidious and initialize our po sei ou odesi trackers
107             status "Detecting"
108             trackers = []
109             status1 "Detecting"
110             trackers1 - []
111
112                 # convert epe egdpth heo a 8100 dpa pass the blob through epa
113                 # peinogk dpa obtain epe detections
114             blob - on2.api. blobFromImage(frame, 0.007843, (N, H), 127.5)
115             net.setInput(blob
116             detections net.Gognaga()
117
118             blob1 cv2.dnn.blobFromImage(frame1, 0.007843, (N, H), 127.5)
119             net1.setInput(blob1
120             detections1 net1.xogamgá()
```

Figure 6.6 Snippet of Program Code

Now we will loop over each of the detections with the hope that we find objects that belong to the category “person.” Looping over the detections in Line 123, we proceed to leverage the confidence (confidence) in Line 126 and filter out weak results and those that do not belong to the category "person" (Lines 130-137).

Now we can bound each person in a box and start tracking correlation. The calculation of the bounding box is performed in Lines 141-142. Next, the correlation tracker of dlib in Line 147, followed by the passing of the coordinates of the object's bounding box to dlib.rectangle, stores the result as correct (Line 148). Next, we start the tracking in Line 149 and add the tracker to the tracking list in Line 153.

```
122                 B loor   over of   de
123 on 1 ep   np.arange(0,  aetostiop.shape[2]):
124                 # extract he said confidence (probability) associated
125                 # with the prediction
126             confidence   detections[0, 0, 1, 2]
127
128                 8 filter not   weak detections hog requiring 8  minimum
129                 8 confidence
130             BUT confidence » 0.4   :
131                     # extract of the exist they   the class label egopi   the
132                     From detections 115E
133                 idx  int(detections[0, 0, 1, 1])
134
135                     # 1E of   class 18061   1:5 where 8   person, lbpogo 1:
136                 12  CLASSES[idx] != "person":
137                         continue
138
139                     # compute epa (ch,  y)-coordinates of Of Voupaiis   box
140                     8 for the object
141                 box  detections[0, 0, 1, 3:7] "pr.   array([W, H, W, H])
142                 (startX, startY, epaX,  endY)  box.astype("int")
143
144                     # construct 8 4110   rectangle odivei egopi   the Roupaiips
145                     # och   coordinates dpa   then start the 4110   correlation
146                     # EgrasKo
147                 tracker  dlib.correlation_tracker()
148                 ge ct 5 4110.  rectangle(startX, startY, epaX, epaY)
149                 ErasKer.                rect)
150
151                     # 864   the tracker to or 115: oi   trackers 50   we can
152                     8 utilize 1t   during skip frames
153                 trackers.append(tracker)
```

Figure 6.7 Program Code Snippet

We repeat the same process for camera 2 as well, but now with the parameters that we have set on camera 2.

```
156 ion 1 apo   np.arange(0, detections1.shape[2]):
157                 # extract epi   confidence (probability) associated
158                 # THE epi   prediction
159             confidence1   detections1[0, 0, 1, 2]
160
161                 # filter ou   weak detections by requiring 8  minimum
162                 8. sopexaepe8? note: non-Greek
163             D^E confidence1 » 6.4   :
164                     # extract the index ou   the class 18661 egopi   the
165                         # detections list
166                 1ak1 -  int(detections1[0, 0, 1, 11)
167
168                     # 1E  the class 18061  to s poe  ha  person, imporre 11
169                 1   CLASSES[idx1] != "person":
170                         eopxapex? (untranslatable)
171
172                     The compute the (ch, y)-coordinates ou   the bounding Ooch
173                     #
174                 box1 - Of the epE   , 0, 1, 3:7] " pri.   array([W, h,  W, H])
175                 in.  odiesex
176
177                     # «opexgysx 8   dlib to ectangle object from the Rupaits
178                     8 och   coordinates apa   then start epo 110   correlation
179                     4 tracker
180                 ERthEkoeRı -  dlib.correlation_trackerThe
181                 rect1 - 4116.  rectangle(startX, startY, epax,  endY)
182                 tracker1.start_track(rgb1, rect1)
183
184                     . add the tracker heo our list of him trackers so we can
185                     4 utilize ex amraps  skip egthpies
186                 trackers1.append(tracker1)
```

Figure 6.8 Excerpt from Program Code

During the skipping of frames, we will use our trackers to monitor our object instead of the detection application.

our trackers for monitoring our object instead of the detection application.

tracking. We begin to browse the available trackers in Line

193. We proceed to update the status to "Tracking" (Line 196) and

we record the position of the object (Lines 199 and 200). From there we extract the

position coordinates (Lines 203-206) and add the information to the list

rects.

```
189                 The otherwise, we should utilize our object *trackers* rather than
190                 # object *detectors* eo  obtain 8 permen eggthepe processing throughput
191         eixe:
192                         # loor  over the trackers
193                 Ton  tracker ap  trackers:
194                             # eei  the s'veyus hoi oung  system eo the  'tracking' rather
195                             # than 'waiting' or 'detecting'
196                         status  "Tracking"
197
198                             # yrapthke epa tracker 8pa vgr  the updated position
199                         tracker.update(rgb)
200                         pos - engvtekken.  _position()
201
202                             # unpack epa  position odheisx
203                         startX  left())
204                         start  - ipiros.  top())
205                         endX - - ipiros.  doxxopi(}})
206                         epa Y X  int(pos.
207
208                             # add epe Boupaipr Och  coordinates he-o the rectangles list
209                         rects.append((startX, startY, epax,  endY))
```

Figure 6.9 Program Code Excerpt

We likewise operate for camera 2, with the sole difference of the

parameters of the corresponding camera.

```
211                 for tracker1 ap  trackers1:
212                             # set the status of our system to d6  'tracking' rather
213                             # than 'waiting' or 'detecting'
214                         status1 = "Tracking"
215
216                             # update epe  tracker and grab the updated position
217                         tracker1.update(rgb1)
218                         pos1 = tracker1.get_position()
219
220                             # unpack the position object
221                         startX = int(pos1.left())
222                         startY = int(pos1.top())
223                         endX = int(pos1.right())
224                         endY = int(pos1.bottom())
225
226                             # add epe  bounding box coordinates to the rectangles list
227                         rects1.append((startX, startY, endX, endY))
228
```

Figure 6.10 Program Code Snippet

In lines 233-234 we create a horizontal line in the output image for each camera. In the parentheses, we define the height, width, and color of each line. Next, in lines 237-238, we use the instance of the CentroidTracker to accept the list of bounding boxes (rects), regardless of whether they were created through object detection or object tracking. Our centroid tracker will associate the recognized objects with object locations.

```
230             # draw ha horizontal 11pth  in the center hoi the frame -- once 8th
231             # object crosses this line we will determine whether they were
232             # moving 'up' or 'down'
233         cv2.line(frame, (0, |  // 2), (W, |  // 2), (o,  0, 200), 2)
234         cv2.line(frame1, (0, |  // 2), (W, |  // 2), (0, 0, 200), 2)
235             # use the centroid tracker to associate the (1) hoia  object
236             # centroids with (2) the newly computed object centroids
237         objects = ct.update(rects)
238         objects1 = ct1.update(rects1)
```

Figure 6.11 Program Code Snippet

Now we will examine the logic that counts whether a person has moved up or down in the frame. We start by looping through the updated coordinates of the bounding box of the recognized objects (Line 240). In line 243 we try to retrieve a TrackableObject for the current object ID. If the TrackableObject does not exist for the object ID, we create one (Lines 246 and 247). Otherwise, there is already an existing one.

TrackableObject, so we need to determine if the object (person) is moving upwards or downwards across the dividing line we set earlier. To do this, we take the value of the y coordinate for all previous centroid positions for the given object (Line 255). Next, we calculate the direction by taking the difference between the current center position and the average of all previous center positions (Line 256). The reason we take the average is to ensure that direction tracking is more stable. If we only stored the previous center position for the person, we would leave open the possibility of measuring the wrong direction, because detection algorithms often predict bounding boxes that may be slightly off from the expected. If the TrackableObject has not been counted (Line 260), we need to determine if it is ready to be counted yet (Lines 264-266) through checking. If the direction is negative (indicating the object is moving upwards) AND the center is above the central line. In this case, we increment the totalUp. Or by checking if the direction is positive (indicating the object is moving downwards) AND the center is below the central line. If this is true, we increment totalDown. Finally, we store the TrackableObject in our trackableObjects dictionary (Line 276), so we can 'grab' it and update it when the next frame is recorded.

```
239                 8 loop oneG epe Egakea odiesis
240         on  (objectID, centroi 4) ep  objects.  exeps():
241                     # check io tes ex 8      trackable odiesx  exists eog epe   current
242                     8. object to
243             To -   trackableObjects.get(objectID, Nops)
244
245                     5 1E  there es po   existing trackable odiesx, egeaie ous
246             1E to es Nope:
247                     To -   TrackableObject(objectID, centroid)
248
249                     5 otherwise, Epege es 8 egvthekvthe odiesx xXo me   can utilize ex
250                     5 to   determine aegeseiop
251             eixe:
252                         4 the difference deineep epe g-eoogaiipaxe oi   the *current*
253                         5: centroid dpa epe   mean ou "rgenious" sepegoias   will te]
254                         8 os Ip niis        ction epe object ex  moving
255                     Y - [e[1]   for s ep to.  centroi 45]
256                     it.  ection - sepegoxa[1] -   np.mean(y)
257                         centroids.append(centroid)
258
259                         5   k io tes 1E   the odiesx pas   been counted or poi
260                     1E poi toi   counted:
261                             5 1E  the direction es   negative
262                             # ANO  the sepegoia es   above epe sepieγ
263                             X pe,   count epe   object
264                         DE  direction " Th dpa   centroid[1] "I // 2:
265                                 totalUp ep 1
266                                 toisoupeea -   True
267
268                             5. if tis aiyeseion   is positive
269                             # ANO epe sepegoxa ex the]on   the
270                             B sepeeg   line, soupe epe odiesx
271                         e11E  direction " 0 dpa   centroid[1] " I // 2:
272                                 totalDown ep 1
273                                 it.  counted   True
274
275                     4 Xioye epe   trackable odiesx ep oug aisxeopdgy
276             trackableObjects[objectID] - to
```

Figure 6.12 Program Code Snippet

In the following code of lines 280-283, we define the characteristics which will have the detection boundaries of the objects. We set the name of each boundary, which will appear, as well as the font, size, and color.

```
278                     # draw both epi IO ou   the odiesx apa epi sepegoia oi tis
279                     # ou]esei oi epi ouxryx egdpe
280             text -i "TO   {}".format(objectID)
281             cv2.putText(frame,  exx  (sepegoxa[0] - 10,   centroid[1] - 10),
282                     cv2.FONT_HEBXHEU XIMRIECH, 0.5, (0, 255, 0), 2)
283             en2.eigeie(egape, (sepegoxa[6],   centroid[1]), 4, (0, 255, 0), -1)
284
```

Figure 6.13 Program Code Snippet

Below we repeat the same process for camera 2.

```
288         for (objectID, centroid) ep  objects1.items():
289                     # check io xee 1E 8     trackable object exists eog epe eunggepe
290                     5  object to
291                 toi -     trackableObjects1.get(objectID, Nops)
292
293                     5. 1E  there es po   existing trackable odiesx, sgeais ope
294                 1E toi     is nope:
295                         toi -     TrackableObject(objectID, centroid)
296
297                     # otherwise, Epege is 8     trackable object "o ne     can utilize ex
298                     # to aexegpipe    direction
299                 eipe:
300                             # tis    difference between epe g-eoogaiipde oi me      *current*
301                             B  centroid apa   the peOn oi "rgenious" sepegoxaxs n111 1611
302                             X hos ep    which aegesxeop epe ou]esei es poniis
303                         y1 - [e[1]    for s ep    to1.centroi45]
304                         direction1 - sepegoxa[1] drrepa(sepegoxa})
305                         to1.centroi pr.pedpig1})
306
307                             # check io Xee ex tes odiesx pas deep        counted og poi
308                         1E poi    to1.counted:
309                                     # 1E the    direction 15 peratine
310                                     5. ANO epe    centroid 1s dyronae    the sepiegy
311                                     5  line, count epe  object
312                                 1E  direction1 "Ho dpa sepegoxa[1] "I // 2:
313                                         totalUp1 += 1
314                                         to1.counted    True
315
316                                     5 16 the    direction es  positive
317                                     # ANO  the centroid es Reion    the
318                                     3 sepiegy    line, count the odiesx
319                                 EPE   direction1 □7 0 dpa  centroid[1] □7 I 7) 2:
320                                         totalDown1 ex 1
321                                         to1.counted X Tgis
322
323                     8 Xioye    the trackable odiesx ep oug    dictionary
324                 trackableObjects1[objectID]    to1
325
```

Figure 6.14 Program Code Snippet

```
326                     # draw both Of the to whom epe ou]esi dpa epe    centroid ou THE
327                     X oriesx op    the ouxryx Egape
328                 teche - "Tr {}".     format(objectID)
329                 cv2.putText(frame1,  Teche,  (centroid[0]  - 10,    centroid[1]  - 10),
330
331                 en2.eige]e(Egapei, HEBXHEY XImri ech, 6.5, (0, 255, 6), 2) (sepegoia[th], 4, (0, 255, 6),
332                 -1)
```

Figure 6.15 Program Code Snippet

We define the variables totalIn, totalOut and Grand_total (Lines 336-338). These variables will be used for the automatic calculation of incoming and outgoing within the surveillance area. Next we set the names of the corresponding variables which will appear in the output images of the two cameras. After setting for camera 1 the names on lines 342-346, we create a display loop for the variables and format the text according to the font, size and color we desire. Likewise we do on lines 356-366 for camera 2. We notice that in camera 1 we define the object movement downward as the 'input', while in the camera

2 we define the 'entry' as the movement of the object upward. This is important to take into account during the installation of the 2 cameras, so that they target the space according to the direction terms we set.

```
334         # Create the 'totalIn' pa´toïθioux        ' which 5eιn   as in the to x81 pyodeG hou reo rιe  going ep on   out
335         # for each camera
336         totalIn=totalDown-totalUp
337         totalOut=totalDown1-totalUp1
338         Grand_total=totalIn-totalOut
339
340         # We set pāpes   on oug   data   which will be sponiēs hop the screen after launch
341
342         apeo t   [
343                  ("Out", totalUp),
344                 (ta,    totalDown),
345                 ("Status", status),
346                 (γtoxdi  EntranceDoor", totalIn), ("agapa Toda] epthe Area " ,  6grapa  _total)]
347
348
349             # lc or  over epe  info tuples dpa agdin  them o oug   r frame
350         To   (i, (k, 9) "Ho:  enumerate(info):
351                 text          {}".format(k, n)
352                 cv2.pu  ame, tech e, (10, H - ((1" 26) 4 20)),
353                          HERSHEY  en2.EONT  XIMRIES, 6.6, (0, 200, 200), 2)
354
355
356         info x  [("In", totalUp1),
357                 {("Owe",totalDown1),
358                 ("Status",status1),("Total ExitDoor",totalOut),
359                 ("Grand Total ep de   Area "', dγθpa  _total)]
360
361
362             # Isor D oneG me epéo        tuples 8pa agon   them o  our egthpe
363         for (i,          in enumerate(info):
364                 XX OS  {}: {}"«Rogpat(k, n)
365                 ext(frame1, text, (10, H - ((1´ 20) « 20)),
366                 cv2.FONT_MEBThis «IMRIEH, 0.6, (0, 200, 200), 2)
367
```

Figure 6.16 Code Fragment

Finally, in lines 370-374, we trigger the display of the two outputs in the form of a pop-up window. We assign the coded names Entrance and ExitDoor respectively. In lines 376-379 we set the 'q' key of the keyboard, as the exit stop key for the outputs. This means that if the q key during program operation, the program will interrupt the exit windows, and thus its operation. In lines 382-383 we add +1 to the number totalFrames we defined previously and refresh the frame rate fps. In line 385 we stop the fps measurement, while in lines 386-387 the command is given to display the running time of the program and the fps rate. Finally, in line 390 the end of recording the cameras is defined and in line 392 we close every open window and thus the operation ends. of the program.

```
369                 5: 5on said ouxryx      frame
370         en2.imshow("Entrance", egthe)
371         KE -    cv2.waitKey(1) a  0xFF
372         cv2.imshow("ExitDoor", Egeipe1)
373         Keg X cv2.waitKey(1) a OchRE
374         Kegi -   cv2.waitKey(1) a THEE
375                 # 1E  the "4 Keg   was rgexthea, Ogeas egopi said loor
376         1E Keg --- oga("a"):
377                         break
375         if key1 xx   ord("q"):
379                         break
380                 # increment Her  total number where Egapes  processed thus Tag  and
381                 BX Epei uradee Her ER5      counter
382         totalFrames ep 1
383         ErXyradte()
384     # Xior  the eidpen dpa   display ER5 epxogpdezxop
385 ErS.etor()
386 rgipE(""“GINEO] Eidreae eipe:   {:.2f}".format(fps.elapsed()))
387 print("[INFO] arrgoch. ER5:   {:.2f}".format(fps.fps()))
388
389     #stop the camera video stream
390 vs.stop()
391     E  close any open windows
392 cv2.destroyAllWindows()
393
```

Figure 6.17 Excerpt of Program Code

6.2 USER GUIDE OF THE PROGRAM

Running the program is quite simple, as long as we have properly installed the environment that was discussed in the previous subchapter. The deliverable folder of the program named ‘People_Counter_Rizos_Konstantinos’ contains 1 executable file and 3 subfolders. The executable file is named ‘People_Counter’, while there are 2 folders named ‘mobilenet_ssd’ and ’ mobilenet_ssd1’ which contain the caffemodel and prototxt for each camera. The pyimagesearch folder contains the center tracking algorithm. The center tracking algorithm detects and tracks the center of an object of interest through many computational blocks. The mobilenet_ssd and mobilenet_ssd1 contain the Caffe model, a Deep Learning model, from which we will use a MobileNet Single Shot Detector (SSD) on each camera.

For running the program, we have chosen the folder People_Counter_Rizos_Konstantinos with the aforementioned files, to be located on the work surface of our computer. It does not matter where it is located, but it will be necessary for the executable to know its exact location within the computer.

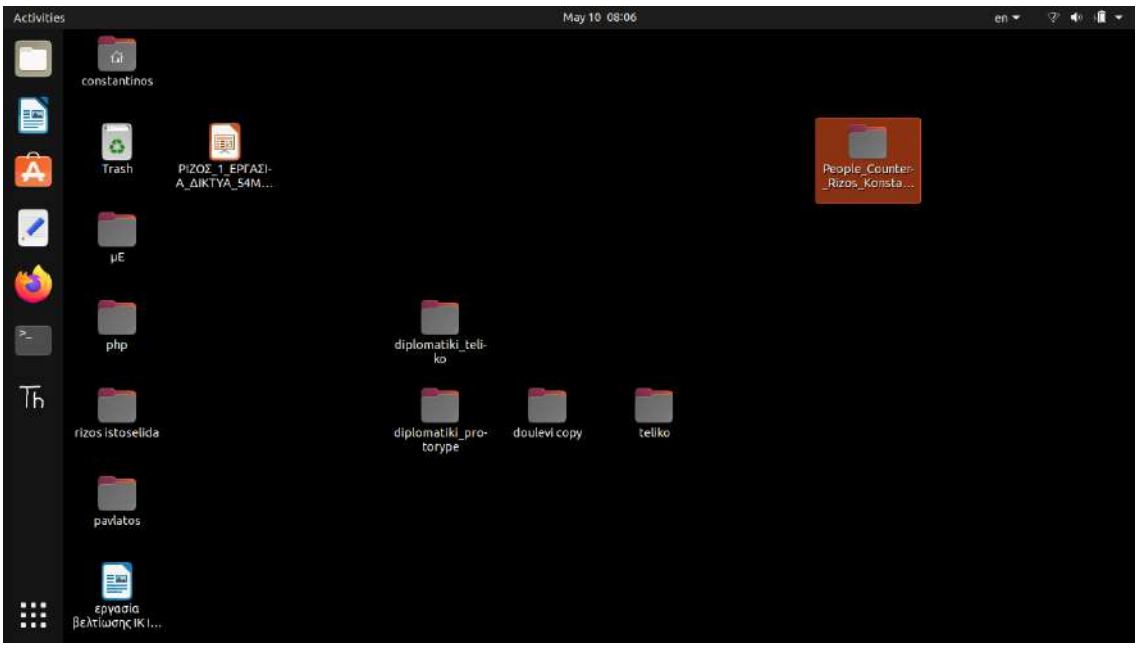

Figure 6.18 Diagram of Program Usage Guide

Next, we will open the Linux terminal as shown below.

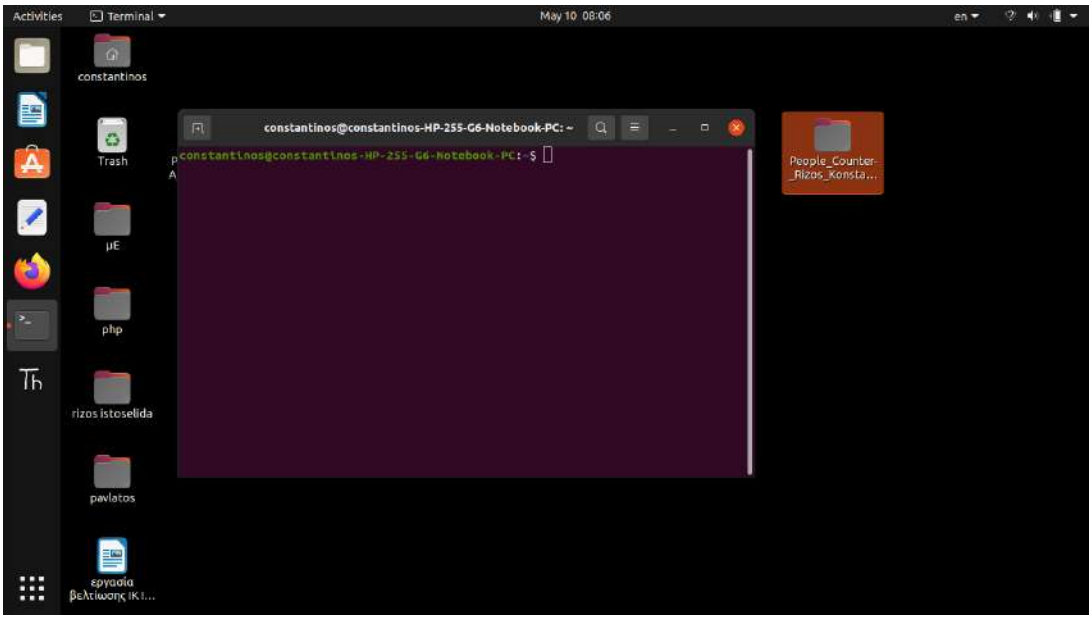

Figure 6.19 Diagram of Program Usage Guide

Then we will enter the command workon cv, where cv (the option name) is a virtual environment in which we have installed the Python..

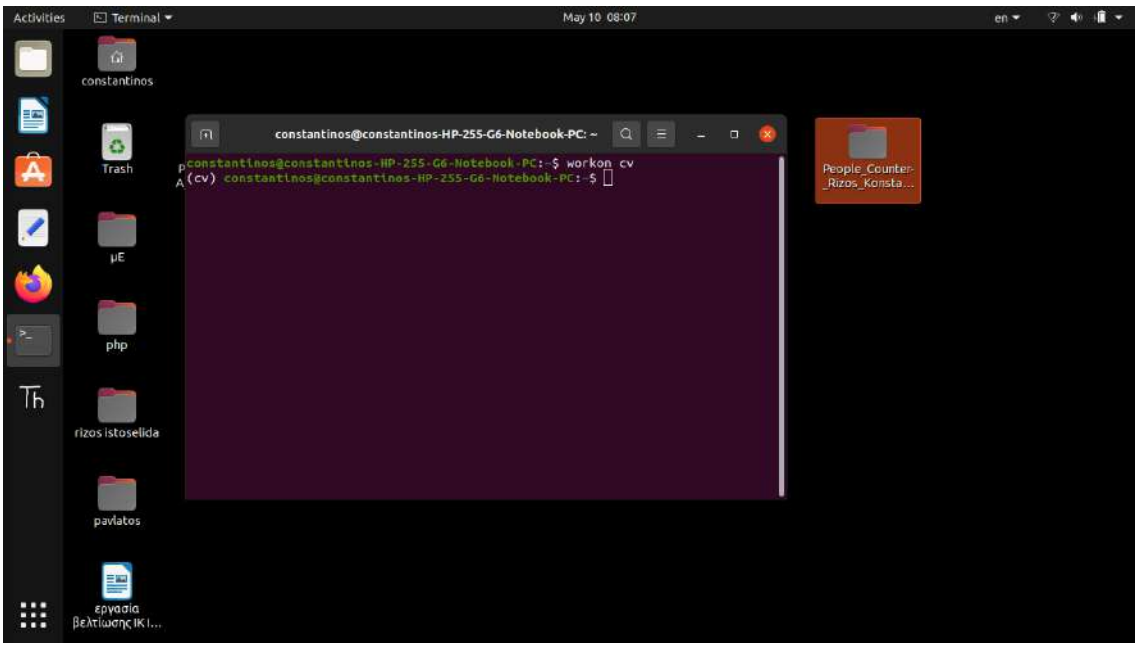

Figure 6. 20 Representation of Program Usage Guide

We notice on the left side of the command line that we are now operating inside the (cv). In the next step we will navigate within the computer, in order to gain access via the terminal to the folder of the program. With the ls command, we can see the contents of the files (the list - list) of the location we are in on the computer.

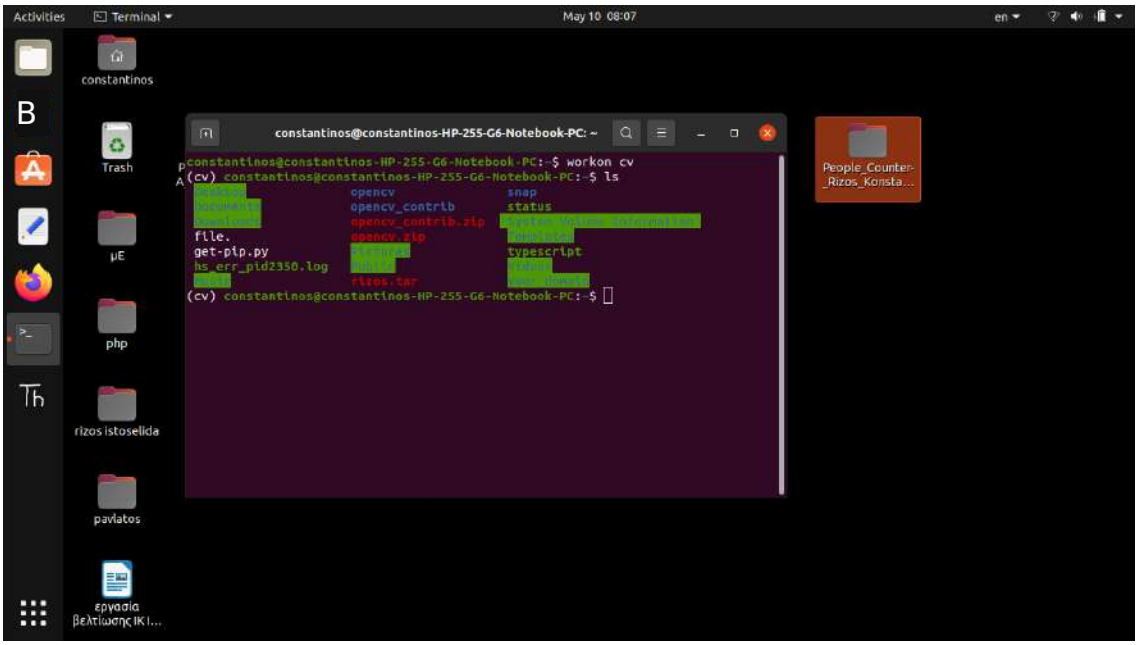

Figure 6. 21 Representation of Program Usage Guide

We use the cd command, where we will be led to the folder that contains the file 'People_Counter.py', which folder is the 'People_Counter_Rizos_Konstantinos'. .

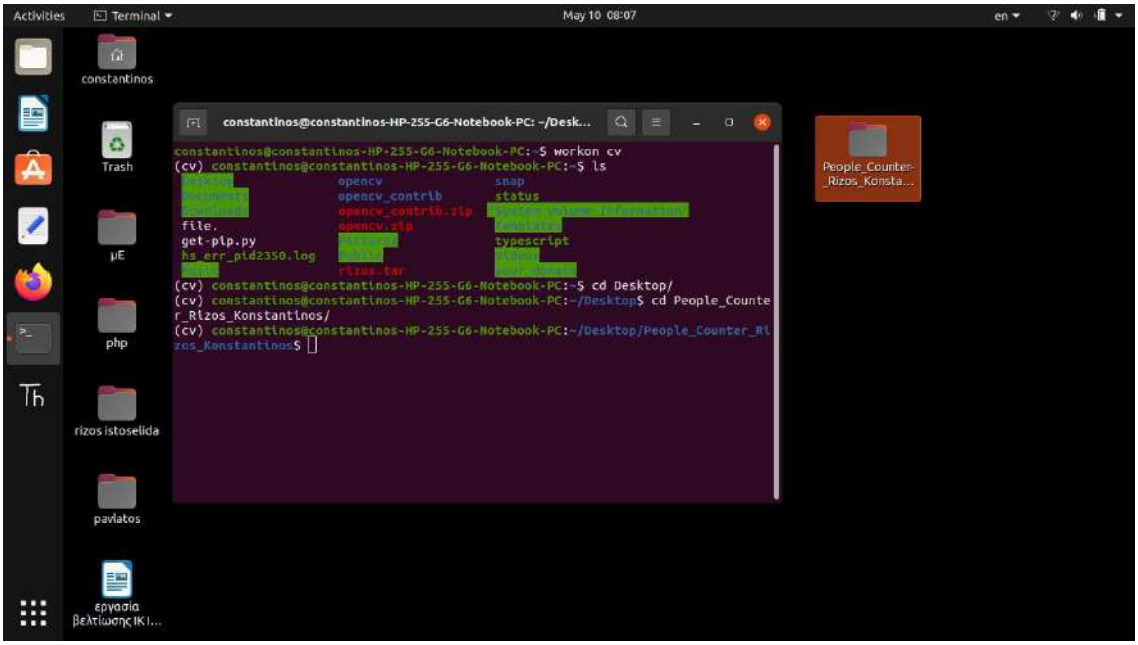

Figure 6.22 Visualization of Program User Guide

With the ls command again, we can confirm that we now are in the correct folder and it contains the files of interest. The location we chose on our computer was the ~/Desktop/People_Counter_Rizos_Konstantinos.

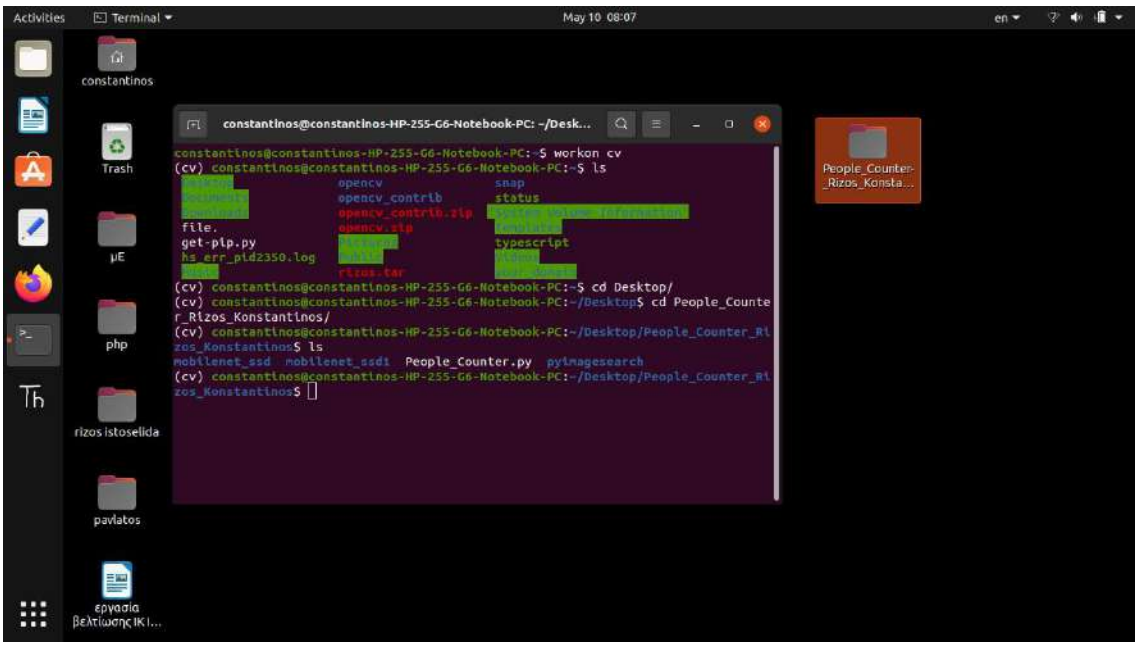

Figure 6.23 Visualization of Program User Guide

Finally, we will give the executable command python People_Counter.py, which instructs the People_Counter.py file to be executed via python, which python is installed in the cv virtual environment. We immediately observe the indications

that notify us about the start of operation of the entry and exit cameras respectively. Next, two separate windows will open, which will display the image from the entrance (Entrance Door) and exit (Exit Door) cameras.

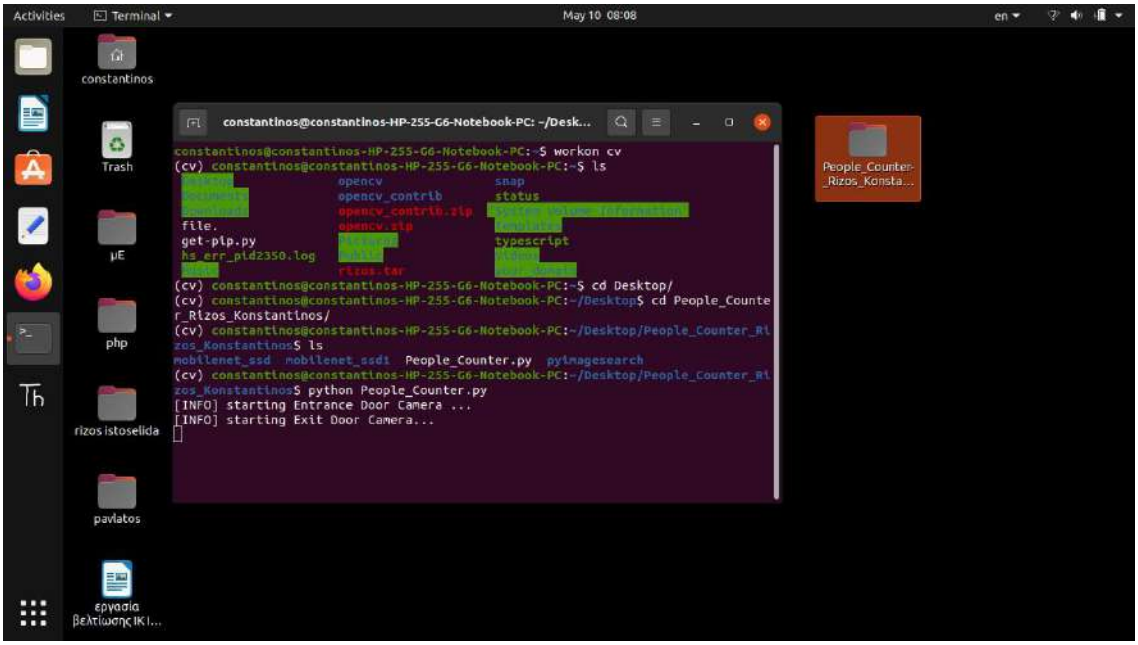

Figure 6. 24 Program User Guide Illustration

6.3 SYSTEM OUTPUT DISPLAY

Below we will see some screenshots from the entry and exit cameras. We observe on the entry camera that it is in an initial state with 0 incoming and 0 outgoing. The door we observe leads out of the area under surveillance, therefore as soon as a person is detected moving from bottom to top, of the red dividing line, it will recognize him as an exit.

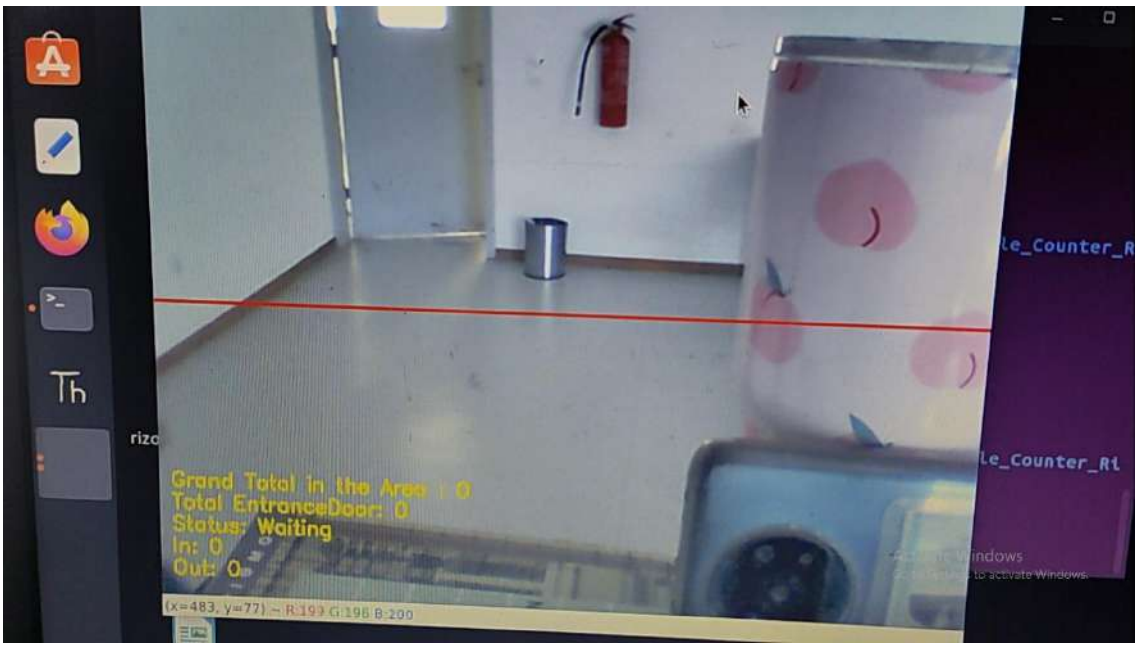

Figure 6. 25 Program Operation Presentation

In the following two images we observe on the left side of the camera how a person appears. It receives ID 0 and once it crosses the red separator line, the Out value takes value 1. The total number inside the space and the total number of the entry door take the value -1.

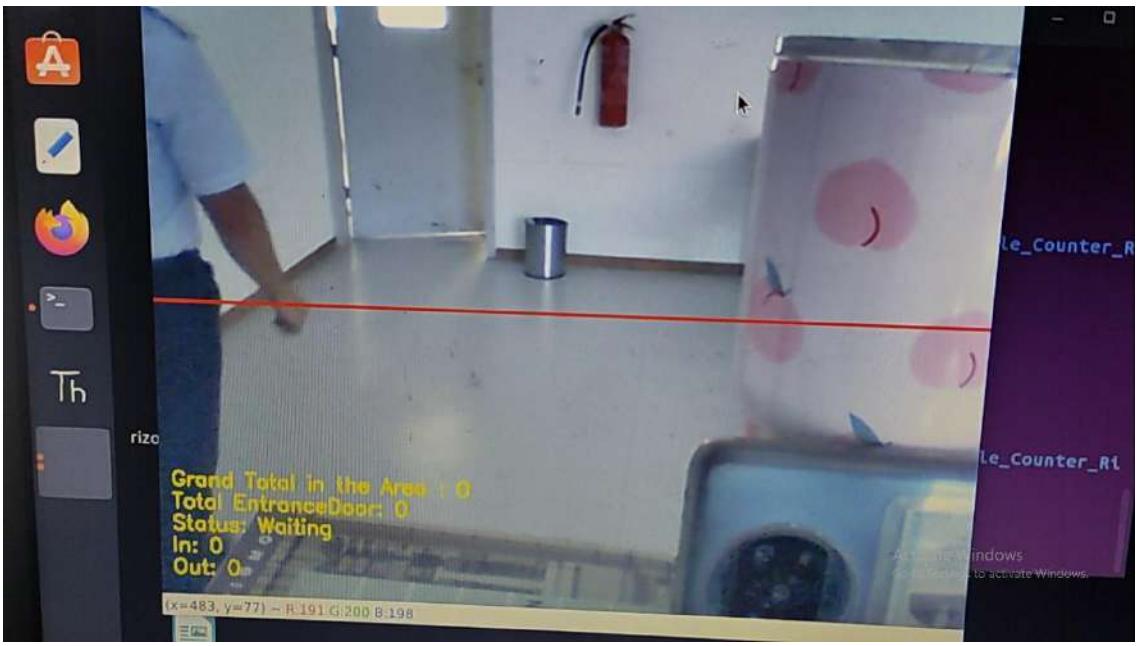

Figure 6. 26 Program Operation Presentation

Figure 6.27 Program Operation Presentation

A few moments later, upon the entry of a person who receives the ID 6, we have a new count and the value of entrance In increases by one unit, as well as the Grand Total in the Area and Total EntranceDoor. Due to the complex configuration of the space, poor lighting, and not high resolution camera and computer, it is likely that errors will be detected. These can be overcome naturally by improving the factors mentioned above.

For example, in the second image, when the entrant crosses the line, the program momentarily freezes ID 6 at a point of the door and remains stationary until it disappears. On the other hand, the entrant receives a new ID, ID 7, which is the one that will cross the line and be recorded. ID 6 is a dead spotting point and does not affect the operation and result of the program.

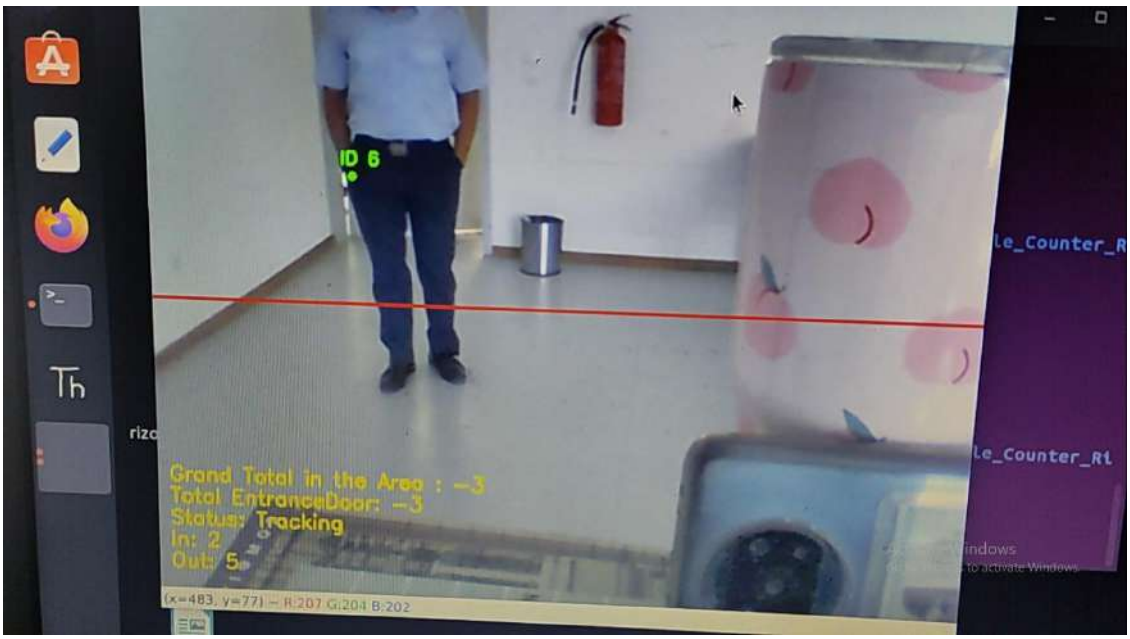

Figure 6.28 Program Operation Presentation

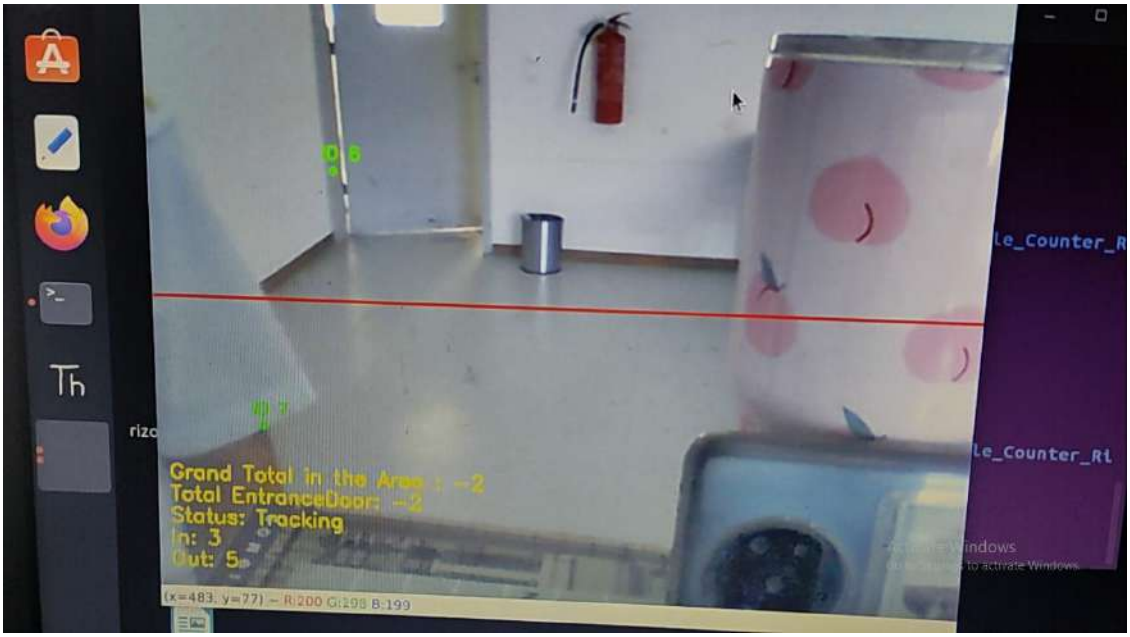

Figure 6.29 Program Operation Presentation

Next, we will see the exit camera Exit Door which is shown below. Note that in this case, a movement of an ID from bottom to top, will be treated as an input into the area.

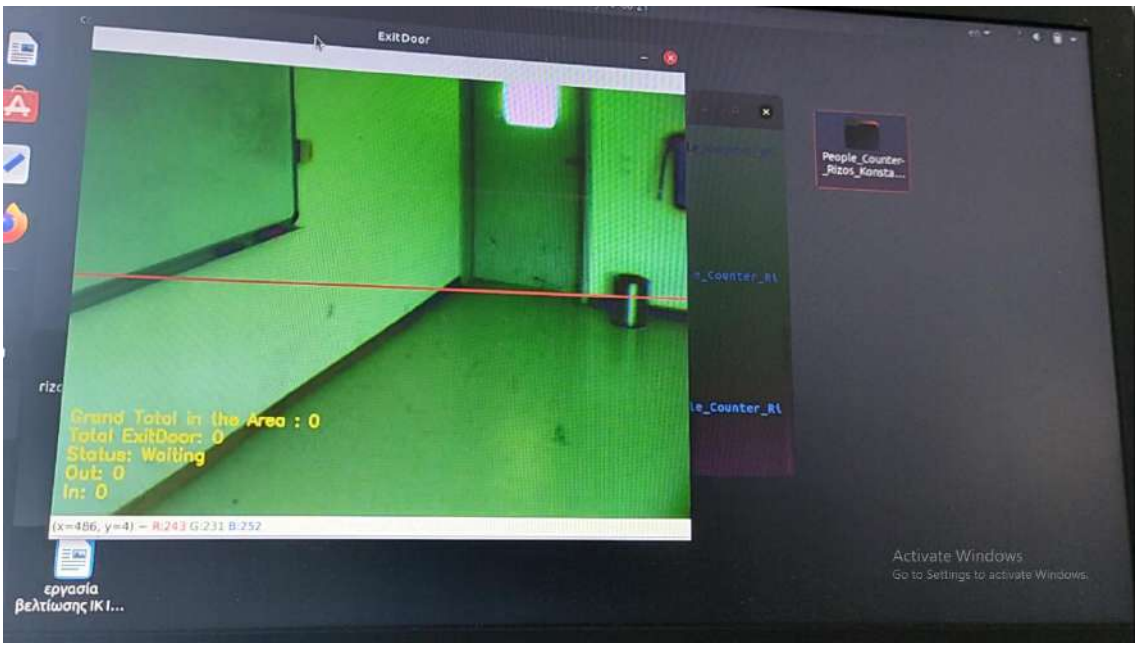

Figure 6. 30 Program Operation Presentation

We see a person moving against the direction of the exit door and will take ID 0, which will be considered as an input to the area. Thus the In and Grand Total will take the value 1, but the total value of the exit will take the value -1, since the exit is considered a positive value.

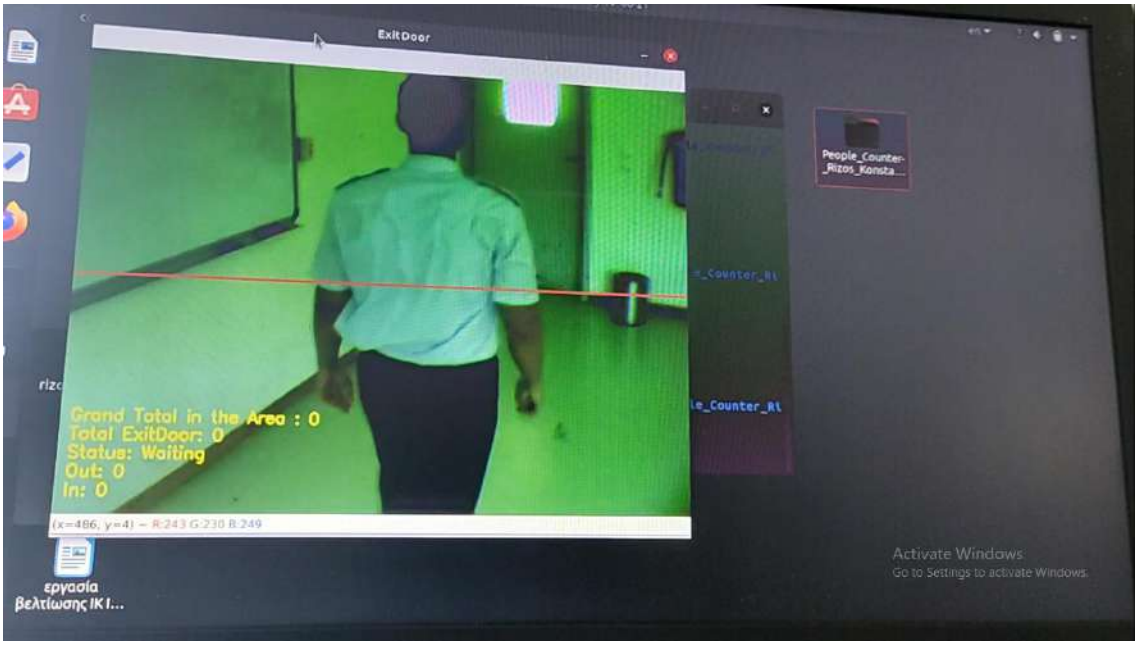

Figure 6. 31 Program Operation Presentation

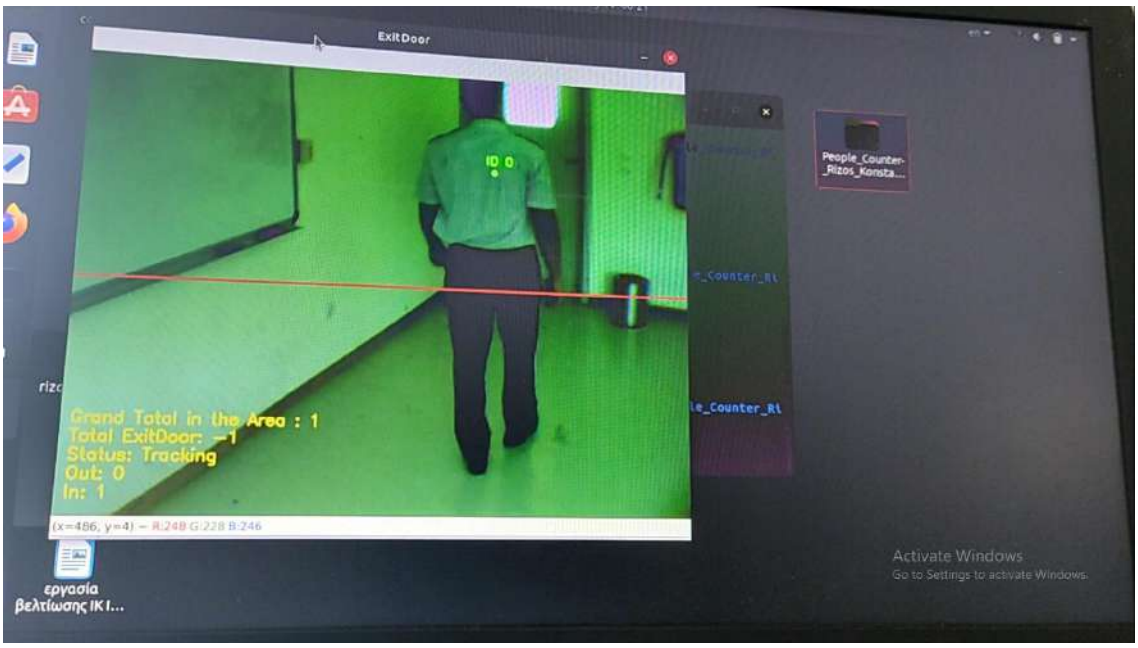

Figure 6.32 Program Operation Presentation

A little while later, after three people have entered the room, we observe one person leaving and receiving the ID value 2. Once it crosses the line, the movement this results in the exit of a person from the space and the counting of it.

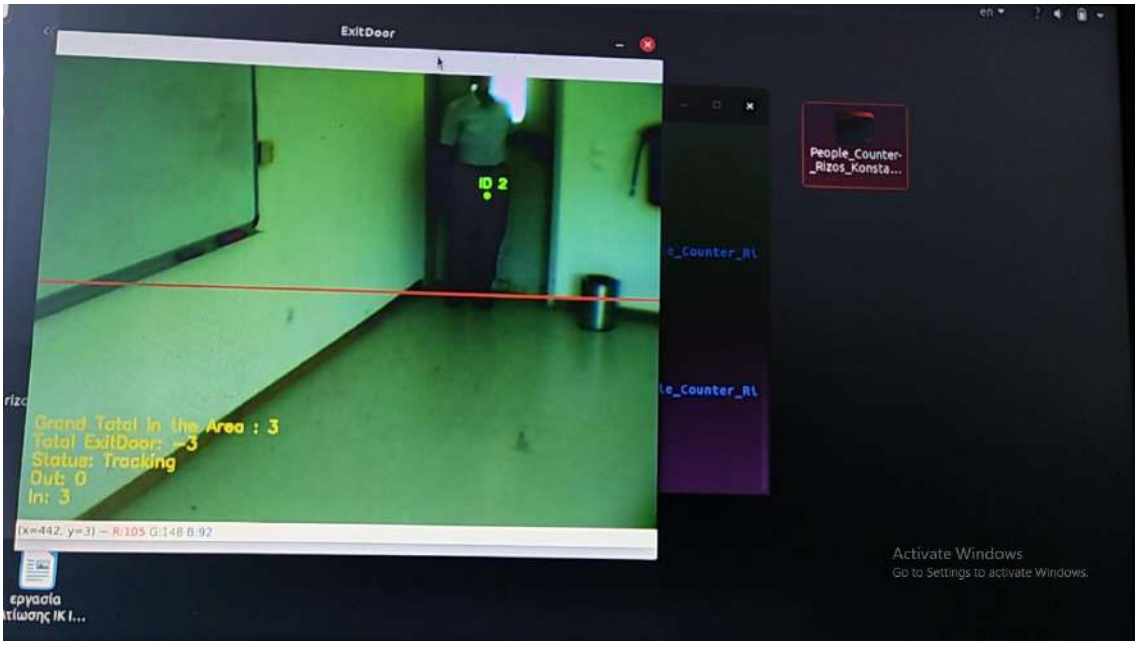

Figure 6.33 Program Operation Presentation

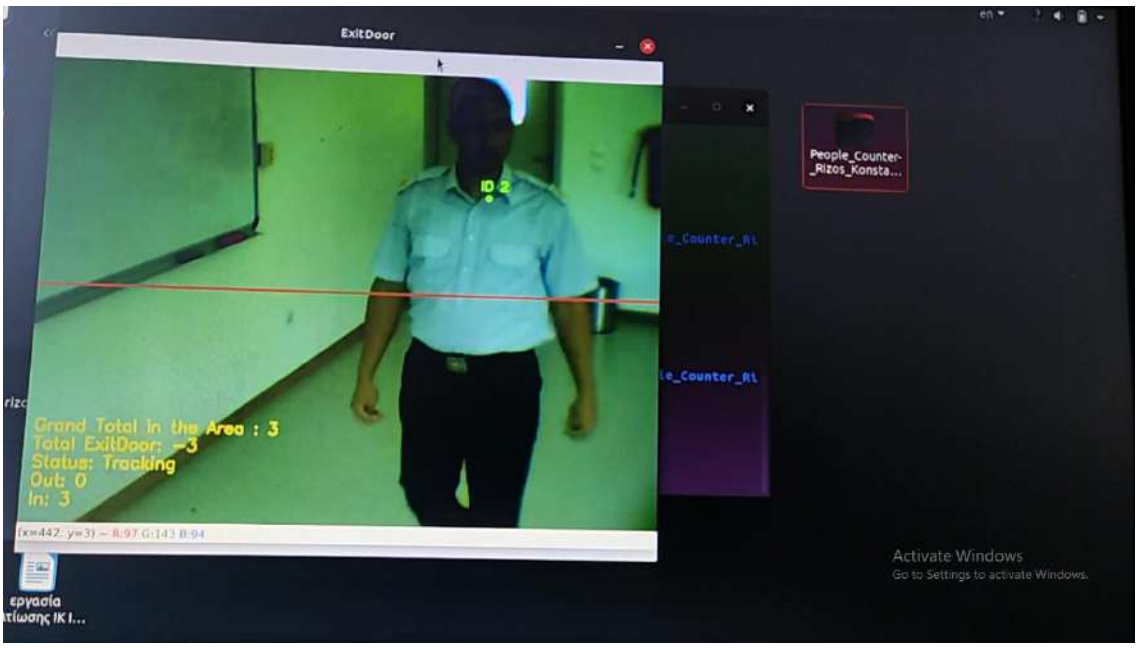

Figure 6. 34 Program Operation Presentation

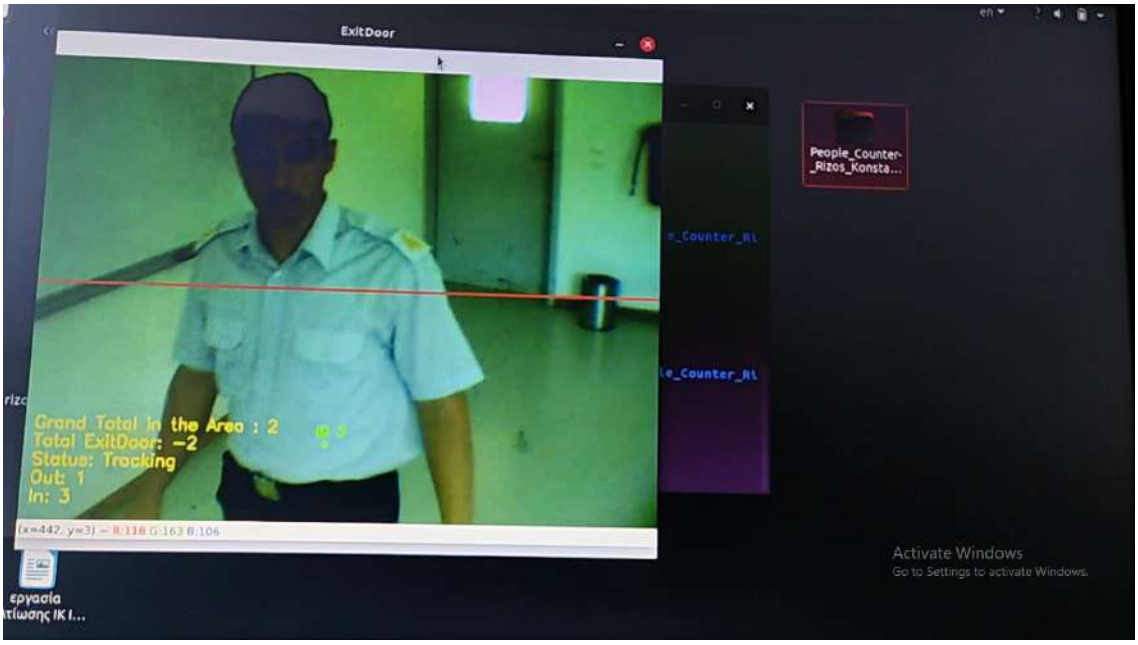

Figure 6. 35 Program Operation Presentation

SEVENTH CHAPTER

CONCLUSIONS – FUTURE APPLICATIONS

7.1 CONCLUSIONS

From the application we just observed, we can confidently say how decisive a factor the evolution of technology and its means is for humanity. The enormous contribution of informatics is obvious, especially when it is used for the common good and of course by people who have the necessary knowledge to develop applications and programs that will solve everyday problems.

Specifically, in our development of the human presence detection program and automatic space monitoring, we solved a daily problem that has arisen from the need to protect the health of the global population. We managed, through simple and basic knowledge, to automate a process, which as we saw in many stores and shops, was carried out by human personnel. This process, apart from being difficult, painful and dangerous for the health of the employee who counted hundreds of customers daily, was an expense for businesses, since they would have to hire at least one more employee. The deeper goal of the program consisted of two important factors which we wanted to coexist so that we could consider the application feasible and efficient. The two factors were, of course, high performance combined with low cost of system construction. The approach to the problem started with the development of the Raspberry system, but it proved to be economically unviable and of very low performance, so we abandoned it. The study continued with the help of a simple and affordable laptop computer, as it turned out to be not only an economical system construction, but also very efficient. Thus, we chose the second method.

7.2 FUTURE APPLICATIONS

The application of this specific program could easily be

The application of the program could be easily developed and expanded in its range of functions. More specifically, regarding the coronavirus, it could include a thermal camera that would perform simultaneous temperature measurement of the person along with their detection. Individuals with temperature above the normal level would be excluded and a rejection indicator would be activated. Also, there could be an entrance or entrances indicator showing when a space has a occupancy level to inform those entering. If someone ignores this warning, an alarm would be triggered to inform the store/venue. To avoid the possibility of ignoring occupancy, automatic entry/exit doors could be connected if present, and in the case of occupancy, they would not allow entry into the monitored area.

Additionally, alternative applications could be installed in

military facilities, in order to verify at the gates of the camp the number of people inside the area. This would ensure that no one could enter without being checked. Of course, the program would need to be updated to detect the number of people in case they enter with a vehicle. The entrance cameras could capture a snapshot of the image every time an entering person is detected, so that if needed, the gate's purpose can consult the snapshots and identify the person(s) of interest.

The system we built has wide applicability in hospitals,

banks, prisons, borders of countries, etc. It is an automated system of cameras that has the ability to adapt according to the needs of each surveillance space. In our case, the need was simple counting of people to combat the spread of the virus. However, as we saw above, it can be adapted to almost any space and serve as a safety, monitoring, and supervision barrier.

1) Sebastian Raschka & Vahif Mirjalili, 2019, Python Machine Learning, Published by Packt Publishing Ltd.

2) Josh Patterson & Adam Gibson, 2017, Deep Learning: A Practitioner's Approach, Published by O'Reilly Media Inc.

3) Subhash Challa, Mark R. Morelande, Darko Musicki, Robin J. Evans, 2011, Fundamentals of Object Tracking, Published in the United States of America by Cambridge Press

4) Jan Erik Solem, 2012, Programming Computer Vision with Python, Published by O'Reilly Media

5) Richard Szeliski, 2011, Computer Vision: Algorithms and Applications, Published by Springer London Dordrecht Heidelberg New York

6) Vlado Damjanovski, 2014, CCTV: From Light to Pixels (Third Edition), Published Elsevier Inc. by

7) Herman Kruegle, 2007, CCTV Surveillance: Analog and Digital Video Practices and Technology (Second Edition), Published by Elsevier Inc.

8) Karman K. and von Brandt A., 1990, Moving object recognition using an adaptive background memory, Published by Elsevier Inc Amsterdam

9) Sean McManus and Mike Cook, 2013, Raspberry Pi for Dummies, A Wiley Brand

10) Simon Monk, 2013, Programming the Raspberry Pi: Getting Started with Python, Published by McGraw-Hill Companies

11) Zed Shaw, 2014, Learn Python the Hard Way (Third Edition), Published by Addison-Wesley

12) David Beazley & Brian K. Jones, 2013, Python Cookbook, Published by O'Reilly Media Inc.

APPENDIX I

SYSTEM CODE

```
# IK IV RIZOS KONSTANTINOS : Program that automatically counts people inside an area

# First we import the necessary packages

from pyimagesearch.centroidtracker import CentroidTracker

from pyimagesearch.trackableobject import TrackableObject

from imutils.video import VideoStream

from imutils.video import FPS

from multiprocessing import Manager

from multiprocessing import Process

import numpy as np

import argparse

import imutils

import time

import dlib

import cv2

import os

import sys


# Insert the list of class labels that MobileNet SSD was trained to detect

CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat",

"bottle", "bus", "car", "cat", "chair", "cow", "diningtable",

"dog", "horse", "motorbike", "person", "pottedplant", "sheep",

"sofa", "train", "tvmonitor"]


# Next we construct the argument parse and parse the arguments for each camera

args = {"prototxt" : "mobilenet_ssd/MobileNetSSD_deploy.prototxt", "model" : "mobilenet_ssd/MobileNetSSD_deploy.caffemodel"

        ,"prototxt1" : "mobilenet_ssd1/MobileNetSSD_deploy.prototxt", "model1" : "mobilenet_ssd1/MobileNetSSD_deploy.caffemodel",
```

```
        "confidence" : 0.4 , "skip-frames" : 30 }

# We set Grand_total equal to zero 0
Grand_total=0

# load our serialized model from disk two times , for each camera
net = cv2.dnn.readNetFromCaffe(args["prototxt"], args["model"])
net1 = cv2.dnn.readNetFromCaffe(args["prototxt1"], args["model1"])

# open camera 1 and camera 2
print("[INFO] starting Entrance Door Camera ...")
vs = VideoStream(src=0).start()
time.sleep(2.0)
print("[INFO] starting Exit Door Camera...")
vs1 = VideoStream(src=2).start()
time.sleep(2.0)

# initialize the frame dimensions
W = None
H = None

# instantiate our centroid tracker, then initialize a list to store
# each of our dlib correlation trackers, followed by a dictionary to
# map each unique object ID to a TrackableObject
ct = CentroidTracker(maxDisappeared=40, maxDistance=50)
trackers = []
trackableObjects = {}

ct1 = CentroidTracker(maxDisappeared=40, maxDistance=50)
trackers1 = []
trackableObjects1 = {}
```

```
# initialize the total number of frames processed thus far, along
# with the total number of objects that have moved either up or down
totalFrames = 0
totalDown = 0
totalUp = 0
totalDown1 = 0
totalUp1 = 0

# start the frames per second throughput estimator
fps = FPS().start()

# loop over frames from the video stream
while True:
        # grab the next frame and handle if we are reading from either
        # VideoCapture or VideoStream
    frame = vs.read()
    frame1 = vs1.read()

        # resize the frame to have a maximum width of 700 pixels
        # then convert
        # the frame from BGR to RGB for dlib
    frame = imutils.resize(frame, width=700)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    frame1 = imutils.resize(frame1, width=700)
    rgb1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)


        # if the frame dimensions are empty, set them
```

```
if W is None or H is None:
        (H, W) = frame.shape[:2]
if W is None or H is None:
        (H, W) = frame1.shape[:2]

    # initialize the current status along with our list of bounding
    # box rectangles returned by either (1) our object detector or
    # (2) the correlation trackers
status = "Waiting"
rects = []
status1 = "Waiting"
rects1 = []



    # check to see if we should run a more computationally expensive
    # object detection method to aid our tracker
if totalFrames % 30 == 0:
            # set the status and initialize our new set of object trackers
        status = "Detecting"
        trackers = []
        status1 = "Detecting"
        trackers1 = []

            # convert the frame to a blob and pass the blob through the
            # network and obtain the detections
        blob = cv2.dnn.blobFromImage(frame, 0.007843, (W, H), 127.5)
        net.setInput(blob)
        detections = net.forward()

        blob1 = cv2.dnn.blobFromImage(frame1, 0.007843, (W, H), 127.5)
        net1.setInput(blob1)
```

```
detections1 = net1.forward()

    # loop over the detections
for i in np.arange(0, detections.shape[2]):
            # extract the confidence (probability) associated
            # with the prediction
        confidence = detections[0, 0, i, 2]

            # filter out weak detections by requiring a minimum
            # confidence
        if confidence > 0.4 :
                    # extract the index of the class label from the
                    # detections list
                idx = int(detections[0, 0, i, 1])

                    # if the class label is not a person, ignore it
                if CLASSES[idx] != "person":
                        continue

                    # compute the (x, y)-coordinates of the bounding box
                    # for the object
                box = detections[0, 0, i, 3:7] * np.array([W, H, W, H])
                (startX, startY, endX, endY) = box.astype("int")

                    # construct a dlib rectangle object from the bounding
                    # box coordinates and then start the dlib correlation
                    # tracker
                tracker = dlib.correlation_tracker()
                rect = dlib.rectangle(startX, startY, endX, endY)
                tracker.start_track(rgb, rect)
```

```
            # add the tracker to our list of trackers so we can
            # utilize it during skip frames
        trackers.append(tracker)

 # loop over the detections
for i in np.arange(0, detections1.shape[2]):
        # extract the confidence (probability) associated
        # with the prediction
    confidence1 = detections1[0, 0, i, 2]

        # filter out weak detections by requiring a minimum
        # confidence
    if confidence1 > 0.4 :
            # extract the index of the class label from the
                # detections list
        idx1 = int(detections1[0, 0, i, 1])

            # if the class label is not a person, ignore it
        if CLASSES[idx1] != "person":
                continue

            # compute the (x, y)-coordinates of the bounding box
            # for the object
        box1 = detections1[0, 0, i, 3:7] * np.array([W, H, W, H])
        (startX, startY, endX, endY) = box1.astype("int")

            # construct a dlib rectangle object from the bounding
            # box coordinates and then start the dlib correlation
            # tracker
        tracker1 = dlib.correlation_tracker()
        rect1 = dlib.rectangle(startX, startY, endX, endY)
```

```
                tracker1.start_track(rgb1, rect1)

                    # add the tracker to our list of trackers so we can
                    # utilize it during skip frames
                trackers1.append(tracker1)


    # otherwise, we should utilize our object *trackers* rather than
    # object *detectors* to obtain a higher frame processing throughput
else:
        # loop over the trackers
    for tracker in trackers:
            # set the status of our system to be 'tracking' rather
            # than 'waiting' or 'detecting'
        status = "Tracking"

            # update the tracker and grab the updated position
        tracker.update(rgb)
        pos = tracker.get_position()

            # unpack the position object
        startX = int(pos.left())
        startY = int(pos.top())
        endX = int(pos.right())
        endY = int(pos.bottom())

            # add the bounding box coordinates to the rectangles list
        rects.append((startX, startY, endX, endY))

    for tracker1 in trackers1:
            # set the status of our system to be 'tracking' rather
```

```
            # than 'waiting' or 'detecting'
        status1 = "Tracking"

            # update the tracker and grab the updated position
        tracker1.update(rgb1)
        pos1 = tracker1.get_position()

            # unpack the position object
        startX = int(pos1.left())
        startY = int(pos1.top())
        endX = int(pos1.right())
        endY = int(pos1.bottom())

            # add the bounding box coordinates to the rectangles list
        rects1.append((startX, startY, endX, endY))


    # draw a horizontal line in the center of the frame -- once an
    # object crosses this line we will determine whether they were
    # moving 'up' or 'down'
cv2.line(frame, (0, H // 2), (W, H // 2), (0, 0, 200), 2)
cv2.line(frame1, (0, H // 2), (W, H // 2), (0, 0, 200), 2)
    # use the centroid tracker to associate the (1) old object
    # centroids with (2) the newly computed object centroids
objects = ct.update(rects)
objects1 = ct1.update(rects1)
    # loop over the tracked objects
for (objectID, centroid) in objects.items()
        # check to see if a trackable object exists for the current
        # object ID
    to = trackableObjects.get(objectID, None)
```

```
    # if there is no existing trackable object, create one
if to is None:
        to = TrackableObject(objectID, centroid)

    # otherwise, there is a trackable object so we can utilize it
    # to determine direction
else:
            # the difference between the y-coordinate of the *current*
            # centroid and the mean of *previous* centroids will tell
            # us in which direction the object is moving
        y = [c[1] for c in to.centroids]
        direction = centroid[1] - np.mean(y)
        to.centroids.append(centroid)

            # check to see if the object has been counted or not
        if not to.counted:
                    # if the direction is negative
                    # AND the centroid is above the center
                    # line, count the object
                if direction < 0 and centroid[1] < H // 2:
                        totalUp += 1
                        to.counted = True

                    # if the direction is positive
                    # AND the centroid is below the
                    # center line, count the object
                elif direction > 0 and centroid[1] > H // 2:
                        totalDown += 1
                        to.counted = True
```

```
            # store the trackable object in our dictionary
        trackableObjects[objectID] = to

            # draw both the ID of the object and the centroid of the
            # object on the output frame
        text = "ID {}".format(objectID)
        cv2.putText(frame, text, (centroid[0] - 10, centroid[1] - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
        cv2.circle(frame, (centroid[0], centroid[1]), 4, (0, 255, 0), -1)

    # construct a tuple of information we will be displaying on the
    # frame

for (objectID, centroid) in objects1.items():
            # check to see if a trackable object exists for the current
            # object ID
        to1 = trackableObjects1.get(objectID, None)

            # if there is no existing trackable object, create one
        if to1 is None:
                to1 = TrackableObject(objectID, centroid)

            # otherwise, there is a trackable object so we can utilize it
            # to determine direction
        else:
                    # the difference between the y-coordinate of the *current*
                    # centroid and the mean of *previous* centroids will tell
                    # us in which direction the object is moving
                y1 = [c[1] for c in to1.centroids]
                direction1 = centroid[1] - np.mean(y1)
                to1.centroids.append(centroid)
```

```
# check to see if the object has been counted or not
if not to1.counted:
    # if the direction is negative
    # AND the centroid is above the center
    # line, count the object
    if direction1 < 0 and centroid[1] < H // 2:
        totalUp1 += 1
        to1.counted = True

    # if the direction is positive
    # AND the centroid is below the
    # center line, count the object
    elif direction1 > 0 and centroid[1] > H // 2:
        totalDown1 += 1
        to1.counted = True

# store the trackable object in our dictionary
trackableObjects1[objectID] = to1

# draw both the ID of the object and the centroid of the
# object on the output frame
text = "ID {}".format(objectID)
cv2.putText(frame1, text, (centroid[0] - 10, centroid[1] - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
cv2.circle(frame1, (centroid[0], centroid[1]), 4, (0, 255, 0), -1)


# Create the 'totalIn' and 'totalOut' which show us the total number of people going in or out
# for each camera
totalIn=totalDown-totalUp
```

```
totalOut=totalDown1-totalUp1
Grand_total=totalIn-totalOut

# We set names on our data , which will be showing on the screen after
launch

info = [
        ("Out", totalUp),
        ("In", totalDown),
        ("Status", status),
        ("Total EntranceDoor", totalIn), ("Grand Total in the Area " ,
Grand_total)]


   # loop over the info tuples and draw them on our frame
for (i, (k, v)) in enumerate(info):
       text = "{}: {}".format(k, v)
       cv2.putText(frame, text, (10, H - ((i * 20) + 20)),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 200), 2)


info = [("In", totalUp1),
       ("Out", totalDown1),
       ("Status",status1),("Total ExitDoor",totalOut),
       ("Grand Total in the Area " , Grand_total)]


   # loop over the info tuples and draw them on our frame
for (i, (k, v)) in enumerate(info):
       text = "{}: {}".format(k, v)
       cv2.putText(frame1, text, (10, H - ((i * 20) + 20)),
       cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 200), 2)
```

```
        # show the output frame
    cv2.imshow("Entrance", frame)
    key = cv2.waitKey(1) & 0xFF
    cv2.imshow("ExitDoor", frame1)
    key = cv2.waitKey(1) & 0xFF
    key1 = cv2.waitKey(1) & 0xFF
        # if the `q` key was pressed, break from the loop
    if key == ord("q"):
            break
    if key1 == ord("q"):
            break
        # increment the total number of frames processed thus far and
        # then update the FPS counter
    totalFrames += 1
    fps.update()
  # stop the timer and display FPS information
fps.stop()
print("[INFO] elapsed time: {:.2f}".format(fps.elapsed()))
print("[INFO] approx. FPS: {:.2f}".format(fps.fps()))

  #stop the camera video stream
vs.stop()
  # close any open windows
cv2.destroyAllWindows()
```